

SpaDES For Dummies

Guide to the SpaDES modelling toolkit in R

Ceres Barros

Eliot McIntire
Alex Chubaty

Tati Micheletti

2024-07-24

Table of contents

Preface	1
1 Introducing SpaDES with a dummy ecological model	3
1.1 BEFORE SpaDES	3
1.1.1 Setup	4
1.1.2 Species abundance data generator	4
1.1.3 Temperature data generator	6
1.1.4 Data analysis	7
1.2 AFTER SpaDES	8
1.2.1 The control script	8
1.2.2 General module structure: <i>speciesAbundance</i> module .	10
1.2.2.1 Defining the module	10
1.2.2.2 Events and event functions	13
1.2.2.3 Scheduling events	15
1.2.2.4 .inputObjects function	17
1.2.2.5 Additional module functions	19
1.2.3 Creating and adding additional modules: the <i>temper-</i> <i>ature</i> module	20
1.2.4 Modules that depend on other modules: the <i>speci-</i> <i>esTempLM</i> module	24
1.2.5 The workflow	29
1.2.5.1 Workflow setup in a “global” script	29
1.2.5.2 Checking the workflow setup	31
1.2.5.3 Running the workflow	34
1.2.6 Additional notes	35
2 A more realistic example of SpaDES	37

2.1	The example: projecting species distribution shifts under climate change	38
2.2	Module creation and coding	38
2.2.1	Data modules	40
2.2.1.1	<i>speciesAbundanceData</i> module:	40
2.2.1.2	<i>climateData</i> module:	46
2.2.2	Prediction module	53
2.2.3	Additional module scripts and functions	59
2.3	Running the model	61
2.3.1	Workflow set-up	61
2.3.2	Workflow initialisation	67
2.3.3	Workflow runs	68
2.3.3.1	Adding a new climate scenario	71
2.3.3.2	Proposed exercises	75
2.3.3.3	Making use of <code>simList</code> for reporting	76
2.4	Caching	78
2.4.1	Explicitly caching operations	78
2.4.2	Implicit caching of events	79
2.4.3	Controlling caching without changing module code	80
2.5	Best practices	80
2.5.1	Reproducible package installation	80
2.5.2	Protect yourself and others from common mistakes/problems	81
2.5.3	Readable code	82
2.5.4	Module documentation (<code>.Rmd</code> file)	82
2.5.5	Coding for the future	83
2.5.6	Transparent models	83
2.6	Additional notes	84
2.7	Acknowledgements	84
	References	85
	Appendices	87
	A Complete R script to Chapter 1	87
	B Complete R script to Chapter 2	89

List of Figures

1	zenodoBadge	1
1.1	Inputs and outputs in SpaDES: Object A comes from outside of the module (e.g. from an internet URL, from data you have, or from <code>.inputObjects</code>), while Module Z produces object C. Both objects serve as an inputs for Module Y, which in return produce as outputs objects B and D, respectively from objects A and C. As Module Z uses a simple function <i>internally</i> to create object C, it doesn't have any inputs, such as our dummy example.	11
1.2	Diagram of module connections.	32
1.3	Module diagram showing module inter-dependencies with object names.	33
1.4	Final plot of the workflow	34
2.1	Study area within Canada.	66
2.2	Module network diagram.	68
2.3	Module diagram showing module inter-dependencies with object names.	69
2.4	Prediction plots. Input <i>Picea glauca</i> percent cover across the landscape. Note that values are converted to presence/absence.	71
2.5	Prediction plots. Bioclimatic variables under baseline (year 1) and future conditions.	72
2.6	Prediction plots: Raw predicted values of species probability of occurrence under (left to right) baseline climate conditions (first year of simulation), 2021-2040, 2041-2060, 2061-2080 and 2081-2100 climate conditions (second to fifth years of simulation) - using MaxEnt.	73

2.7	Prediction plots: Predictions of <i>Picea glauca</i> presence/absence under (left to right) baseline climate conditions (first year of simulation), 2021-2040, 2041-2060, 2061-2080 and 2081-2100 climate conditions (second to fifth years of simulation) - using MaxEnt.	74
2.8	Adding a new scenario: Predictions of <i>Picea glauca</i> probabilities of presences and presence/absence under (left to right) baseline climate conditions, 2041-2060, and 2081-2100 climate projections under two emission scenarios (SSP 136 and SSP 585, the default) – showing MaxEnt forecasts only.	78

List of Tables

Preface



Figure 1: zenodoBadge

This guide will take you through how to make and link your own modules using **SpaDES** in two examples. Both examples draw on basic uses of statistical models in ecology, notably the relationships between environmental variables and species abundance and presence.

[Chapter 1](#) is very minimal, and uses only dummy data. It is meant to introduce you to the different components of a **SpaDES** module.

[Chapter 2](#) uses real and freely available data, and provides a deeper look into several useful aspects of **SpaDES**, notably caching and spatial data processing.

Full R scripts to execute **SpaDES** workflow examples can be found in the Appendices section.

To install **SpaDES**, please have a look at [SpaDES installation](#), or follow steps at the start of [Chapter 2](#).

Chapter 1

Introducing SpaDES with a dummy ecological model

Let's imagine we want to explore how the relationship between a species' abundance and temperature changes over time. Both the abundance data and the temperature data are being constantly updated by other models, and we want to analyse the relationship between the two iteratively, without needing to manually run a script to account for the newly generated data inputs.

1.1 BEFORE SpaDES

If we use R to develop our species abundance and temperature models in the 'conventional way', we'll probably have i) (the worst case scenario) several scripts that run predictions and data treatment/analysis separately and have to be executed manually, or ii) a long script where everything happens - the predictions and data analysis -, iii) a main script that sources others that do the predictions and analyses.

Option i) is more common when different software are used for different parts of the process (e.g., a model in *C++* generates data that is then analysed in R).

Option ii) is inconvenient because very long scripts make changes and updates to the script - debugging can also be more tiresome.

Option iii) is similar to the SpaDES way of thinking. The difference is that

4CHAPTER 1. INTRODUCING SPADES WITH A DUMMY ECOLOGICAL MODEL

SpaDES defines a standard way of writing different components of a model, or of a modelling framework. This makes changing, updating and sharing code - or modules - easier, as well as swapping and adding modules in a modelling framework.

The example below is so minimal that it is unlikely to show the full benefits of using SpaDES - the same could be accomplished with a fairly short script. However, it introduces the different parts of a module and how to link modules.

[Chapter 2](#) goes a step further and uses real datasets to project a tree species distribution across a landscape in Canada. In [Chapter 2](#), we also introduce SpaDES features that we most commonly use in our work (e.g., caching and spatial data processing) and talk about coding best practices (e.g., code assertions).

1.1.1 Setup

This is what you'd normally do... Install all the packages in some way that you probably didn't write in your scripts and then start your script with loading the packages:

```
## please start from a clean R session
remotes::install_github("ropensci/NLMR") ## you will need this ;)

library(terra)
library(quickPlot)
library(ggplot2)
library(ggpubr)
```

You then create or load a study area raster (ours is a raster template):

```
r <- rast(nrows = 100, ncols = 100, xmin = -50, xmax = 50, ymin = -50, ymax = 50)
```

1.1.2 Species abundance data generator

Our VERY simple species abundance model (in form of a function) generates species abundance rasters that follow a Gaussian distribution:

```
abundance_model <- function(ras, Time) {
  abund_outputs <- list()
  for (t in 1:Time) {
    abund_outputs[[t]] <- NLMR::nlm_mpd(
```

```

    ncol = ncol(ras),
    nrow = nrow(ras),
    resolution = unique(res(ras)),
    roughness = 0.5,
    rand_dev = 100,
    rescale = TRUE,
    verbose = FALSE
  ) |>
  rast()
}
return(abund_outputs)
}

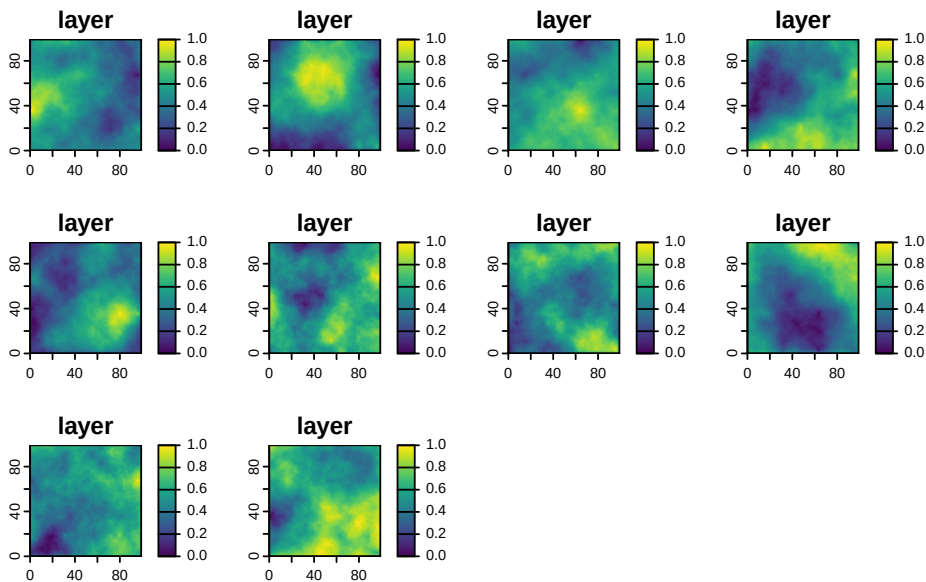
```

Set the number of model iterations, run it and plot results (all ABUNDANCE plots together):

```

Time <- 10
abundance <- abundance_model(ras = r, Time = Time)
dev()
plot(rast(abundance))

```



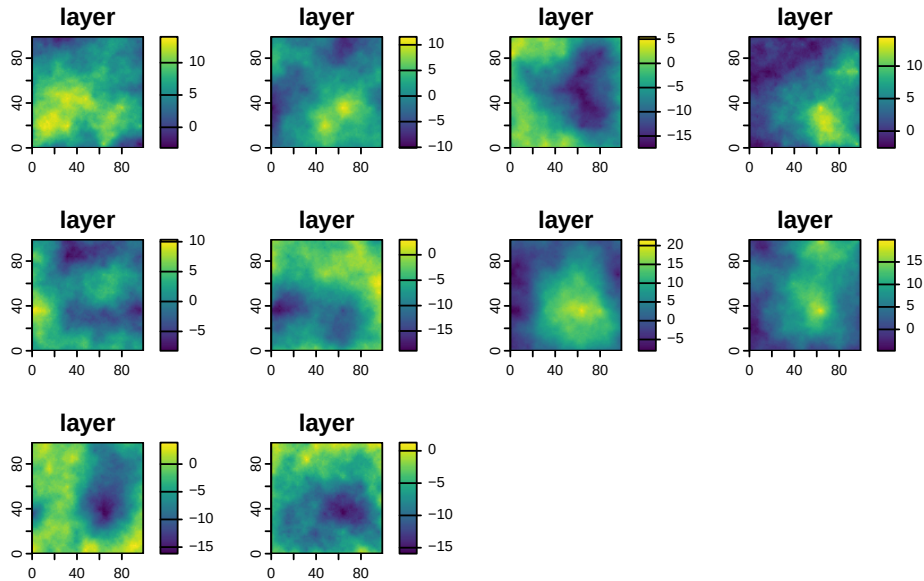
1.1.3 Temperature data generator

The temperature generator model will be similar to the vegetation one - remember this is a dummy example.

```
temperature_model <- function(ras, Time) {
  temp_outputs <- list()
  for (t in 1:Time) {
    temp_outputs[[t]] <- NLMR::nlm_mpd(
      ncol = ncol(ras),
      nrow = nrow(ras),
      resolution = unique(res(ras)),
      roughness = 0.5,
      rand_dev = 10,
      rescale = FALSE,
      verbose = FALSE
    ) |>
      rast()
  }
  return(temp_outputs)
}
```

Run the model and plot results (all temperature plots together)

```
temperature <- temperature_model(ras = r, Time = Time)
plot(rast(temperature))
```



1.1.4 Data analysis

Now we analyse if species abundance and temperature are correlated.

First, we create the data analysis function (a simple linear model):

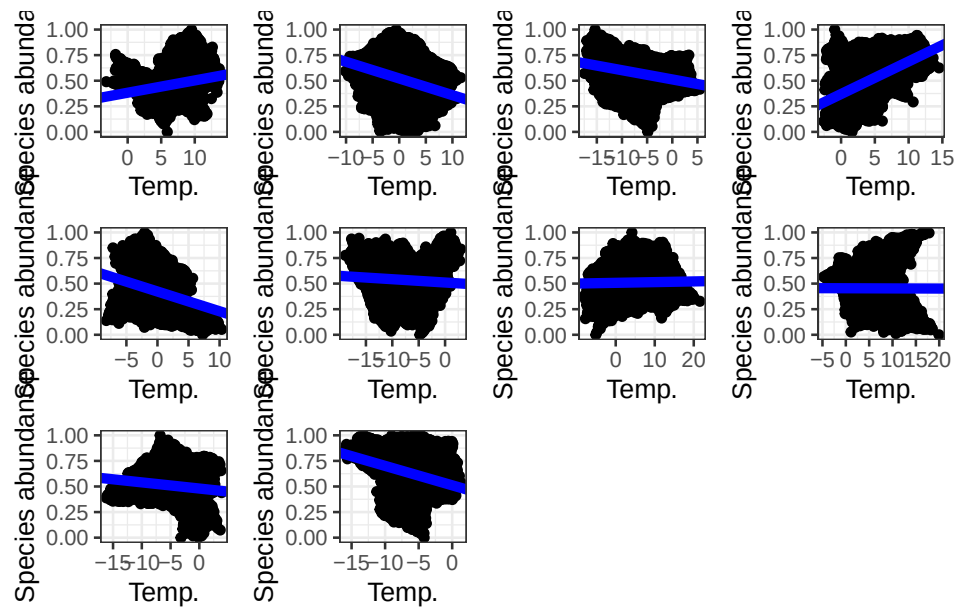
```
stats_analysis <- function(Data) {
  if (all(c("abund", "temp") %in% colnames(Data))) {
    lm1 <- lm(abund ~ temp, data = Data)
    ggplot(Data) +
      geom_point(aes(x = temp, y = abund)) +
      geom_abline(intercept = lm1$coefficients["(Intercept)"],
                  slope = lm1$coefficients["temp"], size = 2, col = "blue") +
      theme_bw() +
      labs(x = "Temp.", y = "Species abundance")
  } else {
    stop("Data must contain 'abund' and 'temp' columns")
  }
}
```

Then we create a loop to analyse each plot of our time-series:

```

lmPlots <- list()
for (t in 1:Time) {
  outputdata <- data.frame(abund = as.vector(abundance[[t]][]), temp = as.vector(
  lmPlots[[t]] <- stats_analysis(Data = outputdata)
}
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2
## 3.4.0.
## i Please use `linewidth` instead.
ggarrange(plotlist = lmPlots)

```



1.2 AFTER SpaDES

1.2.1 The control script

Let us now solve the same problem using the SpaDES approach¹. We start by creating an .R script (it can have any name) that sets up and runs the SpaDES workflow.

The full R script used to run the example can be found in Appendix A.

¹The full R script to run this example Appendix A.

We start by installing `SpaDES.project` which we will use to create the full project structure, install packages and their dependencies are installed, and create the necessary inputs to initialise the workflow.

We also install `SpaDES.core` so that we can use it to create module *template* files.

We then create modules using `newModule`. `newModule` creates a module folder (*speciesAbundance*) inside `/modules` that contains both the module `.R` script template, as well as the documentation template (the `.Rmd` file). Although we will not be discussing the `.Rmd` file, please bear in mind that this is a **fundamental** part of creating a reproducible and transparent module - check out the [Guide to Reproducible Code in Ecology and Evolution](#) from the British Ecological Society). The documentation should contain a the description of the module, its input, parameters and outputs, and potentially a reproducible examples of how the module is executed.

`newModule` also created the folder `/data` where data necessary to the module can be put in, and the folder `/tests` that may contain testing scripts. We will not be using either of them in this example.

```
## start again from a clean R session
options(repos = c("https://predictiveecology.r-universe.dev/",
                  CRAN = "https://cloud.r-project.org"))

install.packages(c("SpaDES.project", "SpaDES.core", "DiagrammeR"))

projPath <- "~/SpaDES4Dummies_Part1"

## create a module template for the species' abundance generator
if (!dir.exists(file.path(projPath, "modules", "speciesAbundance"))) {
  SpaDES.core::newModule(name = "speciesAbundance", path = file.path(projPath, "modules"))
}
```

 Warning

`newModule` should only be run once, otherwise it will replace all edits and contents of the module folder with the templates - this is why it is wrapped in an `if` statement above.

Now go ahead, open the `speciesAbundance.R` script and have a look at it.

1.2.2 General module structure: *speciesAbundance* module

The module template contains all the essential components of a module, with examples, and may seem overwhelming at first. We'll go through it step by step (although not necessarily following the order of the script). The module script can be divided into 4 parts:

i.e., the module's metadata (e.g. module author(s), time units, basic parameters, general inputs and outputs, etc.);

Events and event functions: these are the “actions” (or events) executed in the module (i.e. species reproduction, plotting, saving parameters) - simply put, **WHAT** the module does;

is going to happen (e.g. during the workflow, when will a plot be generated) - simply put, **WHEN** the module does it;

Additional module functions: any additional functions needed (e.g. this is used to keep the coding of your module as clear and straightforward as possible);

The first thing to note is that **the user does not need to manually run** any of the code inside a module's .R script. The function `simInit()` will do so when it sets up the simulation. We will see this later in detail.

1.2.2.1 Defining the module

The first section of the script defines the module's [metadata](#). It allows defining the module's author, keywords, any required packages and module(s) and their versions, but also parameters (and their default values) and input objects that the module requires, and the output objects it creates.

Although this dummy module example requires no true input data, we will define the template raster 'r' as an “input” in the `expectsInput` function, and provide a default object in `.inputObjects` (see below). As for the outputs, it produces a list of abundance rasters (produced during the `abundanceSim` event). So we define it as an output in the `createsOutput` function.

Note that we removed several parameters that come with the template created by the `newModule` function, as they are not needed for this example.

To distinguish what input and output objects are in the context of a module, a good rule of thumb is that inputs are all the `sim$...` objects that appear for the first time (in the module events) on the **right-hand side** of a `<-`, whereas output parameters are the `sim$...` objects that appear for the first time to the **left-hand side** of a `<-`. Another way of explaining it for objects is illustrated in Figure 1.1 :

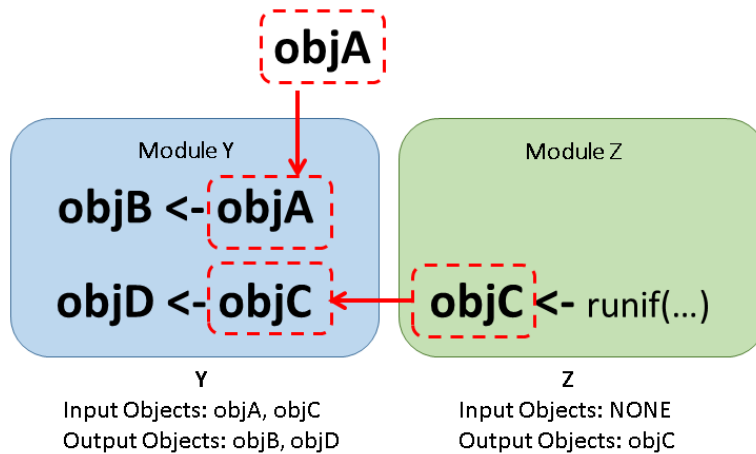


Figure 1.1: Inputs and outputs in SpaDES: Object A comes from outside of the module (e.g. from an internet URL, from data you have, or from `.inputObjects`), while Module Z produces object C. Both objects serve as an inputs for Module Y, which in return produce as outputs objects B and D, respectively from objects A and C. As Module Z uses a simple function *internally* to create object C, it doesn't have any inputs, such as our dummy example.

The default input objects created by the `.inputObjects` function (see `[.inputObjects` function]) during the `simInit` call are exceptions to this rule.

Here is how we defined the *speciesAbundance* module:

```
defineModule(sim, list(
  name = "speciesAbundance",
  description = "",
  keywords = "",
  authors = person("Me", email = "me@example.com", role = c("aut", "cre")),
  childModules = character(0),
  version = list(speciesAbundanceData = "1.0.0"),
```

```

timeframe = as.POSIXlt(c(NA, NA)),
timeunit = "year",
citation = list("citation.bib"),
documentation = deparse(list("README.txt", "speciesAbundance.Rmd")),
reqdPkgs = list("SpaDES.core (>=2.0.2)",
               "terra", "quickPlot"),
parameters = bindrows(
  #defineParameter("paramName", "paramClass", value, min, max, "parameter desc")
  defineParameter("simulationTimeStep", "numeric", 1, NA, NA,
                  "This describes the simulation time step interval"),
  defineParameter(".plotInitialTime", "numeric", 1, NA, NA,
                  "Describes the simulation time at which the first plot event"),
  defineParameter(".plotInterval", "numeric", 1, NA, NA,
                  "Describes the simulation time interval between plot events."),
),
inputObjects = bindrows(
  # expectsInput("objectName", "objectClass", "input object description", source)
  expectsInput("r", objectClass = "SpatRaster", desc = "Template raster")
),
outputObjects = bindrows(
  #createsOutput("objectName", "objectClass", "output object description", ...)
  createsOutput("abundRasters", "list", "List of layers of species abundance at")
)
))

```

Note that the package versions that you define will depend on the ones that are installed on your computer. So take care to change them accordingly. The **SpaDES** package version suggested by the template reflects the version on your computer.

The rest of the script defines the events and their sequences for this module - remember **SpaDES** = Spatial Discrete Event Simulator - and the events themselves.

! Important

defineModule() is not intended to be run directly by the user – it is run internally by the *simInit()* function (see Section 1.2.5.1).

In other words, *you don't run any part of a module's code directly*

in your session; you run `simInit()` with that module listed in the `modules` argument.

1.2.2.2 Events and event functions

Module events are defined and scheduled in the `doEvent.<module name>` function (in this example, `doEvent.speciesAbundance` function; see [Scheduling events]). Since we are only interested in simulating and plotting species abundances, we removed unnecessary events from the script and kept: the initialisation (`init`), an abundance simulation event (`SimulAbund`) and a plotting event (`abundPlot`). Each of these events can execute one or more functions.

Event functions (actual R functions) mustn't be confused with *event names*, which are the names of the events appearing in the `doEvent.<module name>`.

! Important

Event functions take only one argument, `sim` (the `simList` object that stores all objects, modules, functions, etc., of a simulation; see `?SpaDES.core::simList`) and event functions always (and only) return `sim` (using `return(invisible(sim))`).

1.2.2.2.1 Initialisation event function

The initialisation event function (here, `abundanceInit`) can be seen as the starting point of the module. Unlike the `init` event, which must always be present, the function itself does not need to exist (see [Scheduling events]) and can have whatever name we want.

Usually, this function will does pre-simulation steps that are only need to be executed once. In our dummy example, it creates a template raster and a storage list for our species abundance outputs (which will also be rasters). Notice that the only argument to `abundanceInit` is `sim`, a `simList` object that is also its only output.

```
abundanceInit <- function(sim) {
  ## create storage list of species abundance
  sim$abundRasters <- list()
```

```

    return(invisible(sim))
}

```

1.2.2.2.2 *Abundance simulation event function*

The function `abundanceSim` is the core event function of this module, where species abundances are generated via the event. Notice how instead of a *for-loop*, `abundanceSim` runs the `abundance_model` function (which we define separately below) and stores its outputs in the `sim$abundRaster` object. Notice as well that we use `time(sim)` as the identifier of the list slots where outputs are stored (see `?SpaDES.core::time`).

As before, the sole argument and output to this event function is the `sim` object.

```

abundanceSim <- function(sim) {
  ## Generate species abundances - our "simulation"
  sim$abundRasters[[as.character(time(sim))]] <- abundance_model(ras = sim$r)

  return(invisible(sim))
}

```

The `abundanceSim` function was called `Event1` in the template.

1.2.2.2.3 *Plotting event function*

Finally, we created the `abundancePlot` event function to plot the species abundance rasters that are produced by the `abundanceSim` event function. Again, the sole argument and output of this function is `sim`.

```

abundancePlot <- function(sim) {
  ## plot abundances
  plotTitle <- paste("Species abundance\nat time",
                    names(sim$abundRasters)[length(sim$abundRasters)])
  abundPlot <- sim$abundRasters[[length(sim$abundRasters)]]
  Plot(abundPlot,
       title = plotTitle,
       new = TRUE, addTo = "abundPlot")

  return(invisible(sim))
}

```

The `abundancePlot` function was called `plotFun` in the template.

1.2.2.3 Scheduling events

The order in which module events are executed is determined by the `doEvent.<module name>` function. This function also defines the events themselves and what happens in them. The `switch` function executes each event (here `init`, `SimulAbund`, and `abundPlot`) and the events schedule themselves. Two things are of particular importance:

1. The `init` event is **mandatory**. This is the only event whose *name* that cannot be changed and that cannot be removed (even if it does not execute any event functions). All other **events** are optional and can be renamed. `SpaDES` searches and executes all modules' `init` events automatically. Note that the names of event functions executed during `init` *can* have any name: here we changed the `Init` function name (suggested by the template) to `abundanceInit`.
2. **Events should only schedule themselves**. The only exception is the `init`, which schedules the first time all other events are executed (even if a particular event only runs once at the end of the simulation).

It is usually easier to fill the `doEvent.<module name>` function *after* having defined the event functions (as we did above). For instance, we know that plotting should occur after the generation of species abundances, and so the `abundPlot` will be scheduled to occur after the `SimulAbund` event, by changing event priority (see `?priority`).

This is how we configured our `doEvent.speciesAbundance` function:

```
doEvent.speciesAbundance = function(sim, eventTime, eventType, debug = FALSE) {
  switch(
    eventType,
    init = {
      ## do stuff for this event
      sim <- abundanceInit(sim)

      ## schedule future event(s)
      sim <- scheduleEvent(sim, eventTime = start(sim), moduleName = "speciesAbundance",
                           eventType = "SimulAbund")
      sim <- scheduleEvent(sim, eventTime = P(sim)$plotInitialTime,
                           moduleName = "speciesAbundance", eventType = "abundPlot",
```

```

                                eventPriority = .normal()+0.5)
  },
  SimulAbund = {
    ## do stuff for this event
    sim <- abundanceSim(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, eventTime = time(sim) + P(sim)$simulationTimeStep,
                        moduleName = "speciesAbundance", eventType = "SimulAbund",
                        eventPriority = .normal()+0.5)
  },
  abundPlot = {
    ## do stuff for this event
    sim <- abundancePlot(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, eventTime = time(sim) + P(sim)$plotInterval,
                        moduleName = "speciesAbundance", eventType = "abundPlot",
                        eventPriority = .normal()+0.5)
  },
  warning(paste("Undefined event type: '", current(sim)[1, "eventType", with = TRUE],
               "' in module '", current(sim)[1, "moduleName", with = FALSE], '""))
)
return(invisible(sim))
}

```

We suggest having a look at `?base::switch` too fully understand its behaviour. In short, `base::switch` tells R to execute (or switch) different code depending on the value of `EXPR` (here `eventType`). Here, this means that the behaviour of the function `doEvent.speciesAbundance` will change depending on the present `eventType`. So we need to define what behaviour it should have for each event type defined in the module - namely, which functions will be executed and whether to schedule future events with `scheduleEvent`.

1.2.2.3.1 init

The first event is, obviously, `init` - again its **name cannot be changed**.

In `init` we run the initialisation event function (`abundanceInit`) - optional - and schedule the first occurrence of all other events (here, the abundance simulation, `SimulAbund`, and plotting, `abundPlot`, events). Because the

`init` is the only event that `SpaDES` always executes at the start of the simulation, if no events are scheduled during `init`, no events will be executed after the `init`. Notice two things:

1. The `SimulAbund` event is scheduled at `start(sim)` (i.e. at the first time step of the simulation), which means that it will run after the `init` event, but still in the same “year”.
2. `init` schedules the first plotting event to be executed at the time defined by the `.plotInitialTime` parameter, which is stored in the `sim` object (and obtained using `SpaDES.core::P(sim)`), but with a slightly lower event priority `eventPriority = .normal()+0.5` (see `?priority`).

1.2.2.3.2 SimulAbund

The `SimulAbund` event is defined next. This event used to be called `event1` in the template, and we changed its name to be more informative of what it does. It is the core event of this module, where species abundances are generated via the event function `abundanceSim`.

The even also **schedules itself** to occur at a frequency defined by the `simulationTimeStep` parameter

1.2.2.3.3 abundPlot

Finally, we schedule the plotting event, `abundPlot` (which used to be called `plot` in the template). Similarly to the `SimulAbund` event, it executes an event function (`abundancePlot`) and reschedules itself. An important difference is that it uses the `.plotInterval` parameter, instead of `simulationTimeStep`, when rescheduling itself. This way, future events will occur depending on the time step and plot interval parameters defined in the global script (or their default values defined in the metadata section).

1.2.2.4 .inputObjects function

The end of the template `.R` script defines a function called `.inputObjects`. This is where the developer should include code to provide the defaults for any input objects required by the module. This is the ideal place to produce the template raster, `r`, instead of doing so in `abundanceInit`. This will allow a future user (or module) to provide their own `r` object (e.g. for another study area). If on the other hand we don’t do this and create `r` during the

`init` event, any `r` supplied by the user will be overridden by the execution of `init`.

As a rule of thumb, default inputs should be created in a way that allows their values to be overridden by the user (by supplying a named list of objects via `simInit(objects = ...)`) or by any other modules that produce these objects. For this, we rely on the `SpaDES.core::suppliedElsewhere` function, which detects if a given object has already been supplied by the user or if it will be supplied by another module.

Note that `suppliedElsewhere` does not know whether the module that supplies the object will be executed *before* the present module, as it is blind to module scheduling order. When modules are relatively simple and have an approximately linear flow of interdependencies, `SpaDES` is usually able to tell the order in which modules need to be executed. In more complex cases it is a good idea to pass a vector of module names to `simInit(loadOrder = ...)` defining the order of module execution.

Here's an example of how to do this (the commented instructions have been deleted):

```
.inputObjects <- function(sim) {
  if (!suppliedElsewhere("r")) {
    ## make template raster if not supplied elsewhere.
    sim$r <- rast(nrows = 100, ncols = 100, xmin = -50, xmax = 50, ymin = -50, ymax = 50)
  }
  return(invisible(sim))
}
```

If we choose to supply the default `r` in `.inputObjects`, then we need to remove its creation from the `abundanceInit` function and add `r` to the metadata as an input. We have done this, so that `abundanceInit` only creates a storage list for the outputs:

```
abundanceInit <- function(sim) {
  ## create storage list of species abundance
  sim$abundRasters <- list()

  return(invisible(sim))
}
```

It is good practice to provide default input objects to all remaining modules, so that they can work stand-alone. We have done this below.

! Important

If `r` becomes an module input with defaults it must be *added to the module metadata* inside an `expectsInput` call.

1.2.2.5 Additional module functions

Events can also rely on other functions that can either be sourced from other scripts, or defined at the end of the module script (e.g. usually before `.inputObjects`, although the order is irrelevant). This is the case for the species abundances generator function, which we coded in a separate script called `abundance_model.R`. Scripts with accessory functions like these go into module's `R/` folder.

Functions should also be accompanied by metadata. Here we provide a description of the function, its parameters, returning value and what other package functions it relies on using the `roxygen2` documentation style (indicated by `#'`).

```
#' Accessory function to speciesAbundance module
#
#' @param ras a raster layer used as template.
#' @return a fake abundance SpatRaster generated as a Gaussian map with scale = 100 and v
#' @importFrom NLMR nlm_mpd
#' @importFrom terra rast
abundance_model <- function(ras) {
  abund_ras <- NLMR::nlm_mpd(
    ncol = ncol(ras),
    nrow = nrow(ras),
    resolution = unique(res(ras)),
    roughness = 0.5,
    rand_dev = 100,
    rescale = TRUE,
    verbose = FALSE
  ) |>
    rast()
  return(abund_ras)
}
```

1.2.3 Creating and adding additional modules: the *temperature* module

The order in which modules are first executed (i.e. their `init` events) can be automatically determined by inter-module dependencies (i.e. module inputs that are the outputs of other modules). If there are no inter-module dependencies this order is determined by the order in which the modules are listed in the `Part1_DummyModel.R` script, or via the `simInit(loadOrder = ...)` argument.

After the `init` event, the module execution order follows the order of events. This means that a module's events can be scheduled before and after another module's events within the same simulation time step. However, keep in mind that this can make the simulation flow hard to follow, debug and change when additional modules are added.

The second module we created generates yearly temperatures. Apart from different objects and functions names, this module also has the template raster `r` as required input object. Recall that `r` is created during the `.inputObjects` of the *speciesAbundance* module. When the two modules are linked, this object will not be created twice because `suppliedElsewhere("r")` will tell the *temperature* module that 'r' will be supplied by another module. This may appear trivial in this example, but it can be extremely useful when inputs are heavy objects that require lengthy computations to be produced.

This is how we set up the `temperature.R` script looks like:

```
# Everything in this file gets sourced during simInit, and all functions and objects
# are put into the simList.
defineModule(sim, list(
  name = "temperature",
  description = "Temperature simulator",
  keywords = c("temperature", "gaussian", "spatial"),
  authors = structure(list(list(given = c("Ceres"), family = "Barros",
                                role = c("aut", "cre"), email = "ceres.barros@ubc.ca"),
  childModules = character(0),
  version = list(temperature = "1.0.0"),
  timeframe = as.POSIXlt(c(NA, NA)),
  timeunit = "year",
  citation = list("citation.bib"),
```

```

documentation = list("README.txt", "temperature.Rmd"),
reqdPkgs = list("SpaDES.core (>=2.0.2)",
               "terra", "ropensci/NLMR"),
parameters = bindrows(
  #defineParameter("paramName", "paramClass", value, min, max, "parameter description")
  defineParameter("simulationTimeStep", "numeric", 1, NA, NA,
                 "This describes the simulation time step interval"),
  defineParameter(".plotInitialTime", "numeric", 1, NA, NA,
                 "This describes the simulation time at which the first plot event shows up"),
  defineParameter(".plotInterval", "numeric", 1, NA, NA,
                 "This describes the simulation time interval between plot events")
),
inputObjects = bindrows(
  #expectsInput("objectName", "objectClass", "input object description", sourceURL, ...)
  expectsInput("r", "SpatRaster", "Template raster")
),
outputObjects = bindrows(
  #createsOutput("objectName", "objectClass", "output object description", ...),
  createsOutput("tempRasters", "list", "List of raster layers of temperature at any given time")
)
))

## event types
# - type `init` is required for initialization

doEvent.temperature = function(sim, eventTime, eventType, debug = FALSE) {
  switch(
    eventType,
    init = {
      ## do stuff for this event
      sim <- Init(sim)

      ## schedule future event(s)
      sim <- scheduleEvent(sim, eventTime = start(sim), moduleName = "temperature", eventType = "init")
      sim <- scheduleEvent(sim, eventTime = P(sim)$plotInitialTime, moduleName = "temperature",
                          eventType = "tempPlot", eventPriority = .normal() + 0.5)
    },
    SimulTemp = {
      ## do stuff for this event
    }
  )
}

```

```

    sim <- update(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, eventTime = time(sim) + P(sim)$simulationTimeStep,
                        eventType = "SimulTemp")
  },
  tempPlot = {
    ## do stuff for this event
    sim <- plotting(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, eventTime = time(sim) + P(sim)$plotInterval,
                        eventType = "tempPlot", eventPriority = .normal() + 0.001)
  },
  warning(paste("Undefined event type: '", current(sim)[1, "eventType", with = FALSE],
               "' in module '", current(sim)[1, "moduleName", with = FALSE], '")
)
return(invisible(sim))
}

## This is the 'init' event:
Init <- function(sim) {
  ## create storage list of species temperature
  sim$tempRasters <- list()

  return(invisible(sim))
}

## This is the temperature simulation event function
update <- function(sim) {
  ## Generate temperature - our "updated data"
  sim$tempRasters[[as.character(time(sim))]] <- temperature_model(ras = sim$r)

  return(invisible(sim))
}

## This is the plotting event function
plotting <- function(sim) {
  ## plot temperature

```

```

plotTitle <- paste("Temperature\nat time",
                  names(sim$tempRasters)[length(sim$tempRasters)])
tempPlot <- sim$tempRasters[[length(sim$tempRasters)]]
Plot(tempPlot,
     title = plotTitle,
     new = TRUE, addTo = "tempPlot")

return(invisible(sim))
}

.inputObjects <- function(sim) {
  if (!suppliedElsewhere("r")) {
    ## make template raster if not supplied elsewhere.
    sim$r <- rast(nrows = 100, ncols = 100, xmin = -50, xmax = 50, ymin = -50, ymax = 50)
  }
  return(invisible(sim))
}

```

Again, we added an accessory `temperature_model` function in a separate script `R/temperature_model.R`:

```

#' Accessory function to temperature module
#'
#' @param ras a raster layer used as template.
#' @return a fake temperature SpatRaster generated as a Gaussian map with scale = 100 and
#' @importFrom NLMR nlm_mpd
#' @importFrom terra rast
temperature_model <- function(ras) {
  temp_ras <- NLMR::nlm_mpd(
    ncol = ncol(ras),
    nrow = nrow(ras),
    resolution = unique(res(ras)),
    roughness = 0.5,
    rand_dev = 10,
    rescale = FALSE,
    verbose = FALSE
  ) |>
  rast()
  return(temp_ras)
}

```

1.2.4 Modules that depend on other modules: the *speciesTempLM* module

Our third and last module, *speciesTempLM*, will be used to run the statistical analysis at each year, after the abundances and temperatures are generated (**s**pecies and **T**emperature **L**inear **M**odel). Hence, it will depend on the outputs of the *speciesAbundance* and the *temperature* modules.

The interest of keeping the statistical analysis in a separate module lies on the fact that it allows us to easily swap and compare different statistical models to analyse our data if we want to.

It also allows for greater flexibility when it comes to **when** the statistical model is supposed to run. For example, we may want to fit it at every 5 years, instead of every year, using the previous 5 years of data. By having the statistical analysis contained in its own module, we don't need to change other module scripts in order to make these changes.

Finally, we draw your attention to a few differences in this module's script before we see it:

- The **frequency** of the statistical analysis (and correspondent plots) will be determined by the parameter `statsTimestep`. This parameter also determines the number of data years to be used to fit the linear model. If `statsTimestep = 5`, the statistical analysis will use the precedent 5 years of data including the year in which the event is running (a total of 6 years of data);
- This module **requires inputs** that have no defaults in `.inputObjects`. They are specified in `inputObjects` part of `defineModule` - notice how we've respected the names, classes and description of the objects that come from the *speciesAbundance* and the *temperature* modules;
- We have **two additional functions** in a separate script (`R/linear_model_functions.R`): the function fitting the linear model and a plotting function.

Below is the full module script. Notice how the future events where scheduled to `P(sim)$statsTimestep + 0.1`, to force the statistical analyses to occur **after** the abundance and temperature rasters are ready.

```
# Everything in this file gets sourced during simInit, and all functions and obj
```

```

# are put into the simList.
defineModule(sim, list(
  name = "speciesTempLM",
  description = "Statistical analysis of species ~ temperature relationships using LM",
  keywords = c("linear model"),
  authors = structure(list(list(given = c("Ceres"), family = "Barros",
                                role = c("aut", "cre"), email = "ceres.barros@ubc.ca", con

  childModules = character(0),
  version = list(speciesTempLM = "1.0.0"),
  timeframe = as.POSIXlt(c(NA, NA)),
  timeunit = "year",
  citation = list("citation.bib"),
  documentation = list("README.txt", "speciesTempLM.Rmd"),
  reqdPkgs = list("SpaDES.core (>=2.0.2)",
                  "terra", "ggplot2", "data.table", "reshape2"),
  parameters = bindrows(
    #defineParameter("paramName", "paramClass", value, min, max, "parameter description")
    defineParameter("statsTimestep", "numeric", 1, NA, NA, "This describes the how often
  ),
  inputObjects = bindrows(
    #expectsInput("objectName", "objectClass", "input object description", sourceURL, ...)
    expectsInput("abundRasters", "list", "List of raster layers of species abundance at a
    expectsInput("tempRasters", "list", "List of raster layers of temperature at any given
  ),
  outputObjects = bindrows(
    #createsOutput("objectName", "objectClass", "output object description", ...),
    createsOutput("outputdata", "list", "List of dataframes containing species abundances
    createsOutput("outputLM", "list", "List of output yearly LMs (abundance ~ temperature)
    createsOutput("yrs", "numeric", "Vector of years used for statistical analysis")
  )
))

## event types
# - type `init` is required for initialization

doEvent.speciesTempLM = function(sim, eventTime, eventType, debug = FALSE) {
  switch(
    eventType,
    init = {

```

```

    ## do stuff for this event
    sim <- statsInit(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, P(sim)$statsTimestep, "speciesTempLM",
                        "stats", eventPriority = .normal() + 2)
    sim <- scheduleEvent(sim, P(sim)$statsTimestep, "speciesTempLM",
                        "statsPlot", eventPriority = .normal() + 2.5)
  },
  stats = {
    ## do stuff for this event
    sim <- statsAnalysis(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, time(sim) + P(sim)$statsTimestep, "speciesTempLM",
                        "stats", eventPriority = .normal() + 2)
  },
  statsPlot = {
    ## do stuff for this event
    sim <- statsPlot(sim)

    ## schedule future event(s)
    sim <- scheduleEvent(sim, time(sim) + P(sim)$statsTimestep, "speciesTempLM",
                        "statsPlot", eventPriority = .normal() + 2.5)
  },
  warning(paste("Undefined event type: '", current(sim)[1, "eventType", with =
                "' in module '", current(sim)[1, "moduleName", with = FALSE], '
'))
)
return(invisible(sim))
}

## template initialization
statsInit <- function(sim) {
  ## create outputs storage lists
  sim$outputLM <- list()

  return(invisible(sim))
}

```



```

Plot(modelPlot,
      title = plotTitle,
      new = TRUE, addTo = "modelPlot")

return(invisible(sim))
}

.inputObjects <- function(sim) {
  # Any code written here will be run during the simInit for the purpose of creating
  # any objects required by this module and identified in the inputObjects element
  # This is useful if there is something required before simulation to produce the
  # object dependencies, including such things as downloading default datasets, e.g.,
  # downloadData("LCC2005", modulePath(sim)).
  # Nothing should be created here that does not create a named object in inputObjects
  # Any other initiation procedures should be put in "init" eventType of the doEvents
  # Note: the module developer can check if an object is 'suppliedElsewhere' to
  # selectively skip unnecessary steps because the user has provided those inputs
  # simInit call, or another module will supply or has supplied it. e.g.,
  # if (!suppliedElsewhere('defaultColor', sim)) {
  #   sim$map <- Cache(prepareInputs, extractURL('map')) # download, extract, load
  # }

  #cacheTags <- c(currentModule(sim), "function:.inputObjects") ## uncomment this
  dPath <- asPath(getOption("reproducible.destinationPath", dataPath(sim)), 1)
  message(currentModule(sim), ": using dataPath '", dPath, "'.")

  # ! ----- EDIT BELOW ----- ! #

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

```

And the script with the accessory functions:

```

## Accessory functions to speciesTempLM module

#' Accessory function to speciesTempLM module that calculates a
#' linear regression between species abundances and temperature
#'
#' @param Data a data.frame or data.table that contains an \code{abund}

```

```

#'   column and a \code{temp} column with abundance and temperature values
#'   in each location, respectively.
#' @return a linear model (\code{lm}) object fitted with the formula:
#'   \code{abund ~ temp}

linearModel <- function(Data){
  lm1 <- lm(abund ~ temp, data = Data)
  return(lm1)
}

```

1.2.5 The workflow

1.2.5.1 Workflow setup in a “global” script

We can now go back to our `Part1_DummyModel.R` script and set the simulation up.

The function `simInit` needs a few arguments listing simulation folder directories, parameters, simulation times, modules and, optionally, input objects supplied by the user. `simInit` will prepare a simulation object that can later be run by the `spades` function:

- The first list, `modules`, contains modules we want to activate.
- `times` is a named list containing the start and end times of the simulation and what time units we’re working with (with “start” and “end” being the list `names`. It thus defines the length of the simulation. It is important that the start and ending times are defined in decimals, because `SpaDES` allows decomposing time units into smaller fractions.
- `parameters` is a named list of named lists, containing parameters values passed to each module. Note that because the module metadata will (or should) contain default parameter values, here we pass only parameters which we want to change with respect to their defaults. For instance, `.plotInterval` is used and defined in the *speciesAbundance* and *temperature* modules, but not passed to the `simInit` function because we want to use the default value. As a developer providing a reproducible example, we may also chose to list important and useful parameters, even if the value is the same as the default. Here we chose to list `.plotInitialTime` (a parameter used and defined

in the *speciesAbundance* and *temperature* modules), but provide the default value (we experimenting with it by changing its value in the `Part1_DummyModel.R`).

- `paths` contains the folder directory paths that we set earlier.

`setupProject` can be used to create all of these while also making sure all package dependencies are installed. It will also create an [RStudio Project](#) and, if `setupProject(..., Restart = TRUE)` it will open a new RStudio window using that project.

 `setupProject(..., Restart = TRUE)`

We highly recommend using this feature so that an R session is properly started before running a SpaDES workflow. This means that the working directory is set to the root of the project folder, R package libraries (`.libPaths`) are pointing to the right directories and the global environment is as clean as possible (i.e. from pre-loaded packages or any objects)².

```
out <- setupProject(
  paths = list(projectPath = "~/SpaDES4Dummies_Part1",
               packagePath = "packages"),
  Restart = TRUE, ## will open a new RStudio using the newly created RStudio P
  modules = c("speciesAbundance",
              "temperature",
              "speciesTempLM"),
  params = list(speciesAbundance = list(simulationTimeStep = 1,
                                         .plotInitialTime = 1),
                temperature = list(simulationTimeStep = 1,
                                   .plotInitialTime = 1),
                speciesTempLM = list(statsTimestep = 5)),
  times = list(start = 1, end = 10, timeunit = "year")
)
```

Checkout the contents of `out` and see how they correspond to the arguments of `simInit()`:

²Please see **Turn off R diagnostics** and **Turn off previous R session reloading** sections [here](#) for instructions on how to prevent R/RStudio from pre-loading packages and objects.

```

out$paths
out$params
out$times
out$modules

formalArgs(SpaDES.core::simInit)

?SpaDES.core::simInit

```

We now pass the out list entries to `simInit()` arguments

```

## create simList (initialise the workflow)
mySim <- SpaDES.core::simInit(paths = out$paths,
                             modules = out$modules,
                             times = out$times,
                             params = out$params)

```

or, more simply, use the wrapper function `simInit2()`

```

## create simList (initialise the workflow)
mySim <- SpaDES.core::simInit2(out)

```

`simInit` will execute all `.inputObjects` functions, and schedules (but not execute) the `init` events:

```

SpaDES.core::events(mySim)
SpaDES.core::completed(mySim)

```

1.2.5.2 Checking the workflow setup

Before starting the simulations we should check if the modules were linked correctly.

Module diagram

`moduleDiagram` (Figure 1.2) is a useful function that shows module interdependencies as a network diagram. The direction of the arrows indicates an output to input flow. You can see that *speciesAbundance* and *temperature* inputs (specifically our 'r' raster) are supplied by an external source (*INPUT*) - the user or `.inputObjects`. Whereas the inputs to the *speciesTempLM* module are outputs of the *speciesAbundance* and *temperature* modules.

```
SpaDES.core::moduleDiagram(mySim)
```

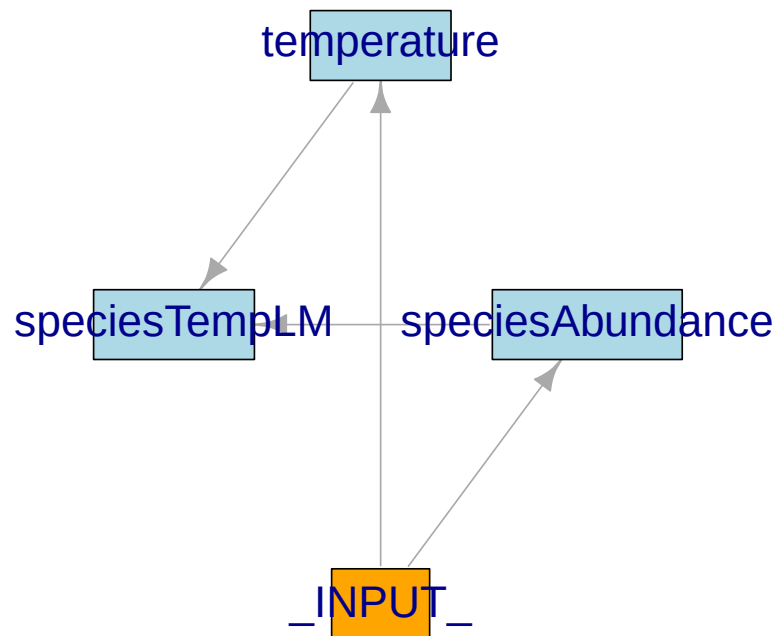


Figure 1.2: Diagram of module connections.

Object diagram

`objectDiagram` (Figure 1.3) provides another way of checking module linkages. It explicitly shows module inter-dependencies by depicting the objects that establish links between modules.

```
SpaDES.core::objectDiagram(mySim)
```

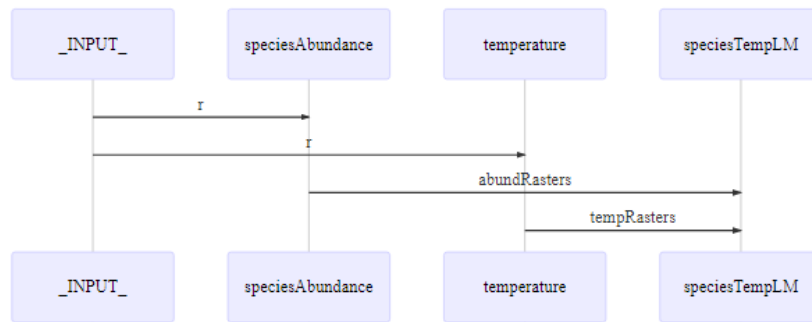


Figure 1.3: Module diagram showing module inter-dependencies with object names.

1.2.5.3 Running the workflow

We run the workflow using the `spades` function.

`spades` output of the `simInit` (the `simList`) and executes the already scheduled `init` events. In turn, these schedule the remainder of the events and `spades` executes them until the end time (`end(sim)`) is reached.

We use `debug = TRUE` so that `spades` prints the events as they are being executed. In case something fails, this helps diagnosing where the issue occurred.

```
## run workflow
dev() # on Windows and Mac, this opens external device if using Rstudio, it is fa
clearPlot()
mySim2 <- SpaDES.core::spades(mySim, debug = TRUE)
```

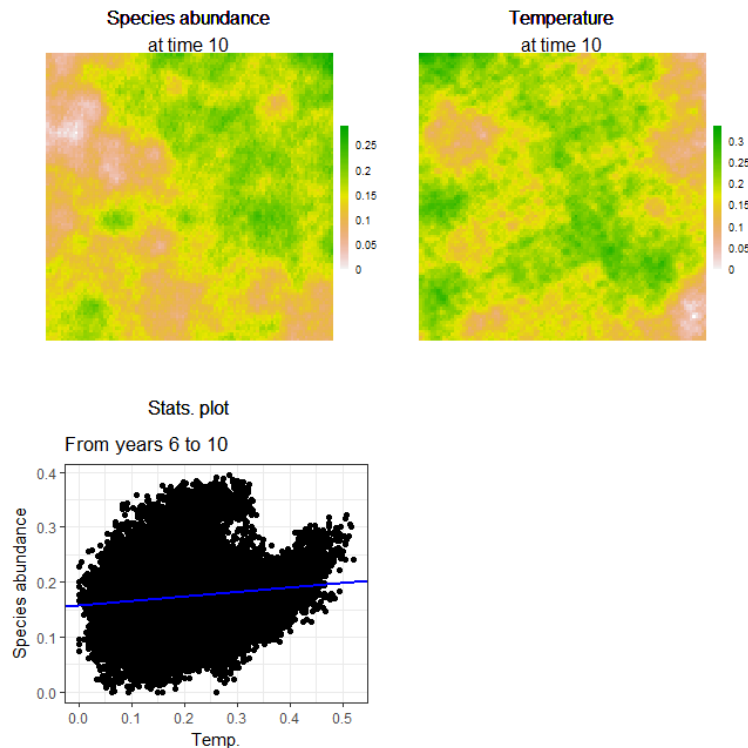


Figure 1.4: Final plot of the workflow

We suggest experimenting with changing parameter values and trying to create and add other modules to further explore all the **SpaDES** flexibility. The more complex the project gets, the more advantageous it is to use **SpaDES** to turn modules *on* or *off*, swapping modules to run, e.g., different statistical analyses, or to include different data.

1.2.6 Additional notes

SpaDES is an extremely powerful family of R packages, and its potential goes well beyond what has been discussed here.

We recommend going to the [SpaDES webpage](#) and the [Robust and nimble scientific workflows, using SpaDES](#) book to learn more.

See also the [Predictive Ecology Community](#) website for news, upcoming SpaDES workshops, a list of packages, and publications, as well as the Predictive Ecology Github repository for all available **SpaDES** modules and **SpaDES**-related packages that we maintain.

Happy SpaDESing!

Chapter 2

A more realistic example of SpaDES

In [Chapter 1](#) of this guide, we described how to create new SpaDES modules, their different components, and how to link them to set up and run a SpaDES workflow.

Here, we assume that you are familiar with these steps, but go further in showing important SpaDES features that facilitate many of the steps common to most ecological modelling exercises.

For the sake of simplicity, we focus our example on projecting a tree species' distribution as a function of climate covariates. Yet, the true power of SpaDES is more evident when using complex dynamic simulation models parametrised using large datasets and ran across large spatial areas (see examples [here](#) and [here](#)).

This Chapter is broken into four main parts: **Module creation and coding**; **Running the model**; **Caching**; and **Best practices**. By no means does it cover caching or best practices in full, as each of these topics is very extensive, but it highlights some of their essentials in SpaDES and from our own experience.

The full R script used to run the example can be found in **Appendix B**.

2.1 The example: projecting species distribution shifts under climate change

Species distribution models (SDMs) have been widely used in ecology to predict how species presences and absences across a landscape may change under changing environmental conditions. As a result, there are several R packages that have been built with this in mind (e.g. `dismo` Hijmans et al. 2021; `biomod2` Thuiller et al. 2021) and many ecologists do these analyses exclusively in R.

Often, these analyses are run only once for a given set of species, baseline and projected environmental conditions, and researchers will have a few R scripts (or maybe just one longer script) that load the data into R, do any necessary pre-processing steps, fit the models and run species distribution projections. The usefulness of `SpaDES` comes when we want an automated and standardized workflow that can be easily updated with new data and adapted with new algorithms. `SpaDES` provides a common standard and a modular approach to modelling that facilitates expanding, debugging and sharing code, but also various tools that bring many well-known best practices from computer- and data-science workflows (including reproducible, modular workflows, and caching), to the realm of ecological modelling, so that they can be used by non-computer-scientists with minimal learning. In an SDM project this means that updating data and algorithms, and automating iterative forecasting become easier and less prone to errors. When `SpaDES` modules are open and shared, this also expands a potential pool of users who can themselves help improve the code.

2.2 Module creation and coding

With the above in mind, in this example we created three modules that source and pre-process data ('data modules') and a module that fits an SDM and iteratively forecasts species distributions (we call it a 'prediction module'). The idea is that we could, for instance, provide different data sources to one of the data modules and only update the parts of the workflow that are affected by this change (i.e. presumably the other data module steps with not be affected). Or, we could develop a second prediction module using a different SDM approach and swap the two modules to inspect which provides better predictions.

Our data modules are *speciesAbundanceData* and *climateData*. The prediction module is *projectSpeciesDist*. We start by creating an .R script to set up and control the workflow. In this example this script is called `Part2_SDMS.R`.

The script begins with a few lines of code to check the R version and install `SpaDES.project` and `SpaDES.core`¹.

Then it generates templates to create a self-contained modules to download and prepare climate and species abundance data.

```
if (getRversion() < "4.2.1") {
  warning(paste("dismo::maxent may create a fatal error",
                "when using R version < v4.2.1 and from RStudio.\n",
                "Please upgrade R, or run this script outside of RStudio.\n",
                "See https://github.com/rspatial/dismo/issues/13"))
}

repos <- c("https://predictiveecology.r-universe.dev/",
          CRAN = "https://cloud.r-project.org")
install.packages(c("SpaDES.project", "SpaDES.core"), repos = repos)

## decide where you're project will be
projPath <- file.path("~/SpaDES4Dummies_Part2")
modPath <- file.path(projPath, "modules")

## create module templates
if (!dir.exists(file.path(modPath, "speciesAbundanceData"))) {
  SpaDES.core::newModule(name = "speciesAbundanceData", path = modPath)
}

if (!dir.exists(file.path(modPath, "climateData"))) {
  SpaDES.core::newModule(name = "climateData", path = modPath)
}

if (!dir.exists(file.path(modPath, "projectSpeciesDist"))) {
  SpaDES.core::newModule(name = "projectSpeciesDist", path = modPath)
}
```

Notice how we protect the `newModule()` calls with an `if` statement that first detects whether the module directory exists already. This is necessary

¹See also [Reproducible package installation](#).

to prevent overwriting existing modules should this script be run a second time in the same location².

Finally, note that we do not load any R packages yet, as we will later use `setupProject` to make sure all module dependencies are installed before running the workflow³.

2.2.1 Data modules

The next two sections show our two data modules .R scripts. We assume you are already familiar with the different parts of a module .R script; if not, see [Chapter 1](#). We do not discuss the module .Rmd files, which should document each module in detail⁴.

2.2.1.1 *speciesAbundanceData* module:

This module downloads freely available spatial layers of *Picea glauca* percent cover (% cover) across Canada and pre-processes them to match a user-supplied study area raster.

The `prepInputs` function downloads the % cover layer from the Canadian National Forest Inventory data server using the URL supplied by `sppAbundURL` and processes it to match the study area raster (`studyAreaRas`) supplied by the user. The module then outputs *Picea glauca* % cover as a raster (`sppAbundanceRas`) and as a `data.table` (`sppAbundanceDT`). The `data.table` contains added information about the year of the workflow during which the data should be used (here, only the first year when SDM fitting happens).

We export species % cover in two formats (a raster and a table) for demonstrational purposes, but also because we could envision that this workflow could output the species distribution projections for several points in time in a more compact format of a `data.table` – large raster layers can consume a considerable amount of disk space⁵.

```
## Everything in this file and any files in the R directory are sourced during `
## all functions and objects are put into the `simList`.
## To use objects, use `sim$xxx` (they are globally available to all modules).
```

²see [Protect yourself and others from common mistakes/problems](#).

³see [Reproducible package installation](#).

⁴See [Section 2.5.4](#).

⁵See [Coding for the future](#).

```

## Functions can be used inside any function that was sourced in this module;
## they are namespaced to the module, just like functions in R packages.
## If exact location is required, functions will be: `sim$.mods$<moduleName>$FunctionName`
defineModule(sim, list(
  name = "speciesAbundanceData",
  description = "",
  keywords = "",
  authors = structure(list(list(given = c("First", "Middle"), family = "Last", role = c("")),
  childModules = character(0),
  version = list(speciesAbundanceData = "0.0.0.9000"),
  timeframe = as.POSIXlt(c(NA, NA)),
  timeunit = "year",
  citation = list("citation.bib"),
  documentation = list("NEWS.md", "README.md", "speciesAbundanceData.Rmd"),
  reqdPkgs = list("SpaDES.core (>= 2.1.5.9001)", "ggplot2"),
  parameters = bindrows(
    #defineParameter("paramName", "paramClass", value, min, max, "parameter description")
    defineParameter(".plots", "character", "screen", NA, NA,
      "Used by Plots function, which can be optionally used here"),
    defineParameter(".plotInitialTime", "numeric", start(sim), NA, NA,
      "Describes the simulation time at which the first plot event should occur"),
    defineParameter(".plotInterval", "numeric", NA, NA, NA,
      "Describes the simulation time interval between plot events."),
    defineParameter(".saveInitialTime", "numeric", NA, NA, NA,
      "Describes the simulation time at which the first save event should occur"),
    defineParameter(".saveInterval", "numeric", NA, NA, NA,
      "This describes the simulation time interval between save events."),
    defineParameter(".studyAreaName", "character", NA, NA, NA,
      "Human-readable name for the study area used - e.g., a hash of the study area name",
      "area obtained using `reproducible::studyAreaName()`"),
    ## .seed is optional: `list('init' = 123)` will `set.seed(123)` for the `init` event
    defineParameter(".seed", "list", list(), NA, NA,
      "Named list of seeds to use for each event (names)."),
    defineParameter(".useCache", "logical", FALSE, NA, NA,
      "Should caching of events or module be used?")
  ),
  inputObjects = bindrows(
    #expectsInput("objectName", "objectClass", "input object description", sourceURL, ...)
    expectsInput(objectName = NA, objectClass = NA, desc = NA, sourceURL = NA)
  )
)

```

```

),
outputObjects = bindrows(
  #createsOutput("objectName", "objectClass", "output object description", ...)
  createsOutput(objectName = NA, objectClass = NA, desc = NA)
)
))

doEvent.speciesAbundanceData = function(sim, eventTime, eventType) {
  switch(
    eventType,
    init = {
      ### check for more detailed object dependencies:
      ### (use `checkObject` or similar)

      # do stuff for this event
      sim <- Init(sim)

      # schedule future event(s)
      sim <- scheduleEvent(sim, P(sim)$plotInitialTime, "speciesAbundanceData",
      sim <- scheduleEvent(sim, P(sim)$saveInitialTime, "speciesAbundanceData",
    },
    plot = {
      # ! ----- EDIT BELOW ----- ! #
      # do stuff for this event

      plotFun(sim) # example of a plotting function
      # schedule future event(s)

      # e.g.,
      #sim <- scheduleEvent(sim, time(sim) + P(sim)$plotInterval, "speciesAbundanceData",

      # ! ----- STOP EDITING ----- ! #
    },
    save = {
      # ! ----- EDIT BELOW ----- ! #
      # do stuff for this event

      # e.g., call your custom functions/methods here
      # you can define your own methods below this `doEvent` function

```

```

    # schedule future event(s)

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + P(sim)$saveInterval, "speciesAbundanceData")

    # ! ----- STOP EDITING ----- ! #
  },
  event1 = {
    # ! ----- EDIT BELOW ----- ! #
    # do stuff for this event

    # e.g., call your custom functions/methods here
    # you can define your own methods below this `doEvent` function

    # schedule future event(s)

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + increment, "speciesAbundanceData", "template")

    # ! ----- STOP EDITING ----- ! #
  },
  event2 = {
    # ! ----- EDIT BELOW ----- ! #
    # do stuff for this event

    # e.g., call your custom functions/methods here
    # you can define your own methods below this `doEvent` function

    # schedule future event(s)

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + increment, "speciesAbundanceData", "template")

    # ! ----- STOP EDITING ----- ! #
  },
  warning(noEventWarning(sim))
)
return(invisible(sim))
}

```

```

### template initialization
Init <- function(sim) {
  # # ! ----- EDIT BELOW ----- ! #

  # ! ----- STOP EDITING ----- ! #

  return(invisible(sim))
}

### template for save events
Save <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event
  sim <- saveFiles(sim)

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

### template for plot events
plotFun <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event
  sampleData <- data.frame("TheSample" = sample(1:10, replace = TRUE))
  Plots(sampleData, fn = ggplotFn) # needs ggplot2

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

### template for your event1
Event1 <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # THE NEXT TWO LINES ARE FOR DUMMY UNIT TESTS; CHANGE OR DELETE THEM.
  # sim$event1Test1 <- " this is test for event 1. " # for dummy unit test
  # sim$event1Test2 <- 999 # for dummy unit test

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

```

```

### template for your event2
Event2 <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # THE NEXT TWO LINES ARE FOR DUMMY UNIT TESTS; CHANGE OR DELETE THEM.
  # sim$event2Test1 <- " this is test for event 2. " # for dummy unit test
  # sim$event2Test2 <- 777 # for dummy unit test

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

.inputObjects <- function(sim) {
  # Any code written here will be run during the simInit for the purpose of creating
  # any objects required by this module and identified in the inputObjects element of def
  # This is useful if there is something required before simulation to produce the module
  # object dependencies, including such things as downloading default datasets, e.g.,
  # downloadData("LCC2005", modulePath(sim)).
  # Nothing should be created here that does not create a named object in inputObjects.
  # Any other initiation procedures should be put in "init" eventType of the doEvent func
  # Note: the module developer can check if an object is 'suppliedElsewhere' to
  # selectively skip unnecessary steps because the user has provided those inputObjects in
  # simInit call, or another module will supply or has supplied it. e.g.,
  # if (!suppliedElsewhere('defaultColor', sim)) {
  #   sim$map <- Cache(prepareInputs, extractURL('map')) # download, extract, load file from
  # }

  #cacheTags <- c(currentModule(sim), "function:.inputObjects") ## uncomment this if Cache
  dPath <- asPath(getOption("reproducible.destinationPath", dataPath(sim)), 1)
  message(currentModule(sim), ": using dataPath '", dPath, "'.")

  # ! ----- EDIT BELOW ----- ! #

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

ggplotFn <- function(data, ...) {
  ggplot2::ggplot(data, ggplot2::aes(TheSample)) +
    ggplot2::geom_histogram(...)

```

```
}
```

2.2.1.2 *climateData* module:

This module downloads and processes freely available spatial layers of four bioclimatic variables used to fit the SDM of *Picea glauca* in the study area.

The module uses a different way to download data. It relies on two input `data.tables` that contain the URLs for each climate covariate, one for baseline conditions, the other for projected climate conditions, both containing information about when each layer should be used during the workflow (the “year” column).

We have only supplied one set of data sources for default baseline climate conditions (`baselineClimateURLs`) and for climate projections (`projClimateURLs`), all of which are downloaded from WorldClim (Fick and Hijmans 2017) at 2.5 minutes resolution. The baseline climate data correspond to the 1970-2000 period, which aligns well with the species % cover data year (2001). The climate projections were obtained for 2021-2040, 2041-2060, 2061-2080 and 2081-2100, from CMIP6 downscaled future projections using the CanESM5 model (Swart et al. 2019) under the SSP 585 emissions scenario.

We encourage providing different (or additional) URLs referring to projections for other climate periods, other climate models and other climate scenarios (go to [WorldClim](#) for a list of climate projections).

When providing other URLs to obtain different climate data, pay special attention to the “year” column of `projClimateURLs`. The URLs need to correspond to the workflow year during which they will be used, not the actual calendar year, unless the workflow years follow the same numbering.

Like in the `speciesAbundanceData` module, the `prepInputs` function processes the climate layers to match the study area raster (`studyAreaRas`) and compiles all climate data in the `climateDT` object and as raster layer objects (`baselineClimateRas` and `projClimateRas`) – the module’s outputs.

```
## Everything in this file and any files in the R directory are sourced during `
## all functions and objects are put into the `simList`.
## To use objects, use `sim$xxx` (they are globally available to all modules).
## Functions can be used inside any function that was sourced in this module;
## they are namespaced to the module, just like functions in R packages.
```

```

## If exact location is required, functions will be: `sim$.mods$<moduleName>$FunctionName`
defineModule(sim, list(
  name = "climateData",
  description = "",
  keywords = "",
  authors = structure(list(list(given = c("First", "Middle"), family = "Last", role = c("
  childModules = character(0),
  version = list(climateData = "0.0.0.9000"),
  timeframe = as.POSIXlt(c(NA, NA)),
  timeunit = "year",
  citation = list("citation.bib"),
  documentation = list("NEWS.md", "README.md", "climateData.Rmd"),
  reqdPkgs = list("SpaDES.core (>= 2.1.5.9001)", "ggplot2"),
  parameters = bindrows(
    #defineParameter("paramName", "paramClass", value, min, max, "parameter description")
    defineParameter(".plots", "character", "screen", NA, NA,
      "Used by Plots function, which can be optionally used here"),
    defineParameter(".plotInitialTime", "numeric", start(sim), NA, NA,
      "Describes the simulation time at which the first plot event should occur"),
    defineParameter(".plotInterval", "numeric", NA, NA, NA,
      "Describes the simulation time interval between plot events."),
    defineParameter(".saveInitialTime", "numeric", NA, NA, NA,
      "Describes the simulation time at which the first save event should occur"),
    defineParameter(".saveInterval", "numeric", NA, NA, NA,
      "This describes the simulation time interval between save events."),
    defineParameter(".studyAreaName", "character", NA, NA, NA,
      "Human-readable name for the study area used - e.g., a hash of the study
      "area obtained using `reproducible::studyAreaName()`"),
    ## .seed is optional: `list('init' = 123)` will `set.seed(123)` for the `init` event
    defineParameter(".seed", "list", list(), NA, NA,
      "Named list of seeds to use for each event (names)."),
    defineParameter(".useCache", "logical", FALSE, NA, NA,
      "Should caching of events or module be used?")
  ),
  inputObjects = bindrows(
    #expectsInput("objectName", "objectClass", "input object description", sourceURL, ...)
    expectsInput(objectName = NA, objectClass = NA, desc = NA, sourceURL = NA)
  ),
  outputObjects = bindrows(

```

```

    #createsOutput("objectName", "objectClass", "output object description", ...)
    createsOutput(objectName = NA, objectClass = NA, desc = NA)
  )
))

doEvent.climateData = function(sim, eventTime, eventType) {
  switch(
    eventType,
    init = {
      ### check for more detailed object dependencies:
      ### (use `checkObject` or similar)

      # do stuff for this event
      sim <- Init(sim)

      # schedule future event(s)
      sim <- scheduleEvent(sim, P(sim)$plotInitialTime, "climateData", "plot")
      sim <- scheduleEvent(sim, P(sim)$saveInitialTime, "climateData", "save")
    },
    plot = {
      # ! ----- EDIT BELOW ----- ! #
      # do stuff for this event

      plotFun(sim) # example of a plotting function
      # schedule future event(s)

      # e.g.,
      #sim <- scheduleEvent(sim, time(sim) + P(sim)$plotInterval, "climateData", "plot")

      # ! ----- STOP EDITING ----- ! #
    },
    save = {
      # ! ----- EDIT BELOW ----- ! #
      # do stuff for this event

      # e.g., call your custom functions/methods here
      # you can define your own methods below this `doEvent` function

      # schedule future event(s)

```

```

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + P(sim)$saveInterval, "climateData", "save")

    # ! ----- STOP EDITING ----- ! #
  },
  event1 = {
    # ! ----- EDIT BELOW ----- ! #
    # do stuff for this event

    # e.g., call your custom functions/methods here
    # you can define your own methods below this `doEvent` function

    # schedule future event(s)

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + increment, "climateData", "templateEvent")

    # ! ----- STOP EDITING ----- ! #
  },
  event2 = {
    # ! ----- EDIT BELOW ----- ! #
    # do stuff for this event

    # e.g., call your custom functions/methods here
    # you can define your own methods below this `doEvent` function

    # schedule future event(s)

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + increment, "climateData", "templateEvent")

    # ! ----- STOP EDITING ----- ! #
  },
  warning(noEventWarning(sim))
)
return(invisible(sim))
}

### template initialization

```

```

Init <- function(sim) {
  # # ! ----- EDIT BELOW ----- ! #

  # ! ----- STOP EDITING ----- ! #

  return(invisible(sim))
}
### template for save events
Save <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event
  sim <- saveFiles(sim)

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

### template for plot events
plotFun <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event
  sampleData <- data.frame("TheSample" = sample(1:10, replace = TRUE))
  Plots(sampleData, fn = ggplotFn) # needs ggplot2

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

### template for your event1
Event1 <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # THE NEXT TWO LINES ARE FOR DUMMY UNIT TESTS; CHANGE OR DELETE THEM.
  # sim$event1Test1 <- " this is test for event 1. " # for dummy unit test
  # sim$event1Test2 <- 999 # for dummy unit test

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

```

```

### template for your event2
Event2 <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # THE NEXT TWO LINES ARE FOR DUMMY UNIT TESTS; CHANGE OR DELETE THEM.
  # sim$event2Test1 <- " this is test for event 2. " # for dummy unit test
  # sim$event2Test2 <- 777 # for dummy unit test

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

.inputObjects <- function(sim) {
  # Any code written here will be run during the simInit for the purpose of creating
  # any objects required by this module and identified in the inputObjects element of def
  # This is useful if there is something required before simulation to produce the module
  # object dependencies, including such things as downloading default datasets, e.g.,
  # downloadData("LCC2005", modulePath(sim)).
  # Nothing should be created here that does not create a named object in inputObjects.
  # Any other initiation procedures should be put in "init" eventType of the doEvent func
  # Note: the module developer can check if an object is 'suppliedElsewhere' to
  # selectively skip unnecessary steps because the user has provided those inputObjects in
  # simInit call, or another module will supply or has supplied it. e.g.,
  # if (!suppliedElsewhere('defaultColor', sim)) {
  #   sim$map <- Cache(prepareInputs, extractURL('map')) # download, extract, load file from
  # }

  #cacheTags <- c(currentModule(sim), "function:.inputObjects") ## uncomment this if Cache
  dPath <- asPath(getOption("reproducible.destinationPath", dataPath(sim)), 1)
  message(currentModule(sim), ": using dataPath '", dPath, "'.")

  # ! ----- EDIT BELOW ----- ! #

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

ggplotFn <- function(data, ...) {
  ggplot2::ggplot(data, ggplot2::aes(TheSample)) +
    ggplot2::geom_histogram(...)

```

```
}
```

We draw your attention to a few particular aspects of the data modules:

- How we took care to define the data classes of parameters, expected inputs and module outputs in their respective metadata sections;
- How we listed all module R packages dependencies to the metadata;
- How we added default values for parameters (metadata) and inputs (`.inputObjects`) explicitly used by the modules. Some parameters like `.plotInterval` were left as `NA`. The exception was the `studyAreaRas` input object for which *we do not provide a default* because we want to force the user to pick a study area. However, we added a code check in `.inputObject` that interrupts the workflow if this object is not in `sim`⁶.
- How we use the function `prepInputs` to do most of the heavy-lifting of downloading data and spatial pre-processing. This function is able to recognize whether the data has already been downloaded, and can cache all spatial processing tasks⁷. In some cases, we wrapped `prepInputs` in a `Map` call to loop through several URLs and download and pre-process many data layers. This `Map` call can also be cached with `Cache`.
- How we export objects created within the module functions to `sim`. Without doing so, these objects are lost after the function is executed and can't be shared between modules.
- How we use the function `Plots` to control plotting to the screen device and/or save to image files depending on the `P(sim)$plots` argument. Note that `Plots` works best with functions that output `ggplot` objects, or that are compatible with `quickPlot::Plot`.
- The fact that neither module depends on the other. This is not a required feature of data modules, but it's good practice to ensure module interoperability.

⁶See [Protect yourself and others from common mistakes/problems](#).

⁷See [Caching](#).

2.2.2 Prediction module

We show below the `.R` script for the `projectSpeciesDist` module. This module depends entirely on the other two, as we did not provide any default input objects in the `.inputObjects` function. This is *not good practice*, but again we warn the user early on (in the `.inputObjects` function) if the module cannot find the necessary inputs.

By default, this module fits a machine learning SDM using the MaxEnt algorithm implemented in the `dismo` package. We recommend having a look at [this guide](#) to learn about fitting SDMs with `dismo` and more. Before fitting the SDM, the module converts any non-binary species data into presences and absences.

This module performs model fitting (i.e., calibration), predictions and model validation. These three components could be broken into three separate modules. As an exercise, we recommend trying to do so on your own.

The main outputs are species distribution projections in the form of plots and a stacked raster layer (`sppDistProj`) and the fitted SDM object.

```
## Everything in this file and any files in the R directory are sourced during `simInit()`
## all functions and objects are put into the `simList`.
## To use objects, use `sim$xxx` (they are globally available to all modules).
## Functions can be used inside any function that was sourced in this module;
## they are namespaced to the module, just like functions in R packages.
## If exact location is required, functions will be: `sim$.mods$<moduleName>$FunctionName`
defineModule(sim, list(
  name = "projectSpeciesDist",
  description = "",
  keywords = "",
  authors = structure(list(list(given = c("First", "Middle"), family = "Last", role = c("author", "maintainer")),
  childModules = character(0),
  version = list(projectSpeciesDist = "0.0.0.9000"),
  timeframe = as.POSIXlt(c(NA, NA)),
  timeunit = "year",
  citation = list("citation.bib"),
  documentation = list("NEWS.md", "README.md", "projectSpeciesDist.Rmd"),
  reqdPkgs = list("SpaDES.core (>= 2.1.5.9001)", "ggplot2"),
  parameters = bindrows(
    #defineParameter("paramName", "paramClass", value, min, max, "parameter description")
    defineParameter(".plots", "character", "screen", NA, NA,
```

```

        "Used by Plots function, which can be optionally used here")
defineParameter(".plotInitialTime", "numeric", start(sim), NA, NA,
        "Describes the simulation time at which the first plot event
defineParameter(".plotInterval", "numeric", NA, NA, NA,
        "Describes the simulation time interval between plot events.")
defineParameter(".saveInitialTime", "numeric", NA, NA, NA,
        "Describes the simulation time at which the first save event
defineParameter(".saveInterval", "numeric", NA, NA, NA,
        "This describes the simulation time interval between save ev
defineParameter(".studyAreaName", "character", NA, NA, NA,
        "Human-readable name for the study area used - e.g., a hash o
        "area obtained using `reproducible::studyAreaName()`")
## .seed is optional: `list('init' = 123)` will `set.seed(123)` for the `ini
defineParameter(".seed", "list", list(), NA, NA,
        "Named list of seeds to use for each event (names)."),
defineParameter(".useCache", "logical", FALSE, NA, NA,
        "Should caching of events or module be used?")
),
inputObjects = bindrows(
  #expectsInput("objectName", "objectClass", "input object description", source
  expectsInput(objectName = NA, objectClass = NA, desc = NA, sourceURL = NA)
),
outputObjects = bindrows(
  #createsOutput("objectName", "objectClass", "output object description", ...)
  createsOutput(objectName = NA, objectClass = NA, desc = NA)
)
))

doEvent.projectSpeciesDist = function(sim, eventTime, eventType) {
  switch(
    eventType,
    init = {
      ### check for more detailed object dependencies:
      ### (use `checkObject` or similar)

      # do stuff for this event
      sim <- Init(sim)

      # schedule future event(s)

```

```

sim <- scheduleEvent(sim, P(sim)$plotInitialTime, "projectSpeciesDist", "plot")
sim <- scheduleEvent(sim, P(sim)$saveInitialTime, "projectSpeciesDist", "save")
},
plot = {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event

  plotFun(sim) # example of a plotting function
  # schedule future event(s)

  # e.g.,
  #sim <- scheduleEvent(sim, time(sim) + P(sim)$plotInterval, "projectSpeciesDist",

  # ! ----- STOP EDITING ----- ! #
},
save = {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event

  # e.g., call your custom functions/methods here
  # you can define your own methods below this `doEvent` function

  # schedule future event(s)

  # e.g.,
  # sim <- scheduleEvent(sim, time(sim) + P(sim)$saveInterval, "projectSpeciesDist",

  # ! ----- STOP EDITING ----- ! #
},
event1 = {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event

  # e.g., call your custom functions/methods here
  # you can define your own methods below this `doEvent` function

  # schedule future event(s)

  # e.g.,

```

```

    # sim <- scheduleEvent(sim, time(sim) + increment, "projectSpeciesDist", "t
    # ! ----- STOP EDITING ----- ! #
  },
  event2 = {
    # ! ----- EDIT BELOW ----- ! #
    # do stuff for this event

    # e.g., call your custom functions/methods here
    # you can define your own methods below this `doEvent` function

    # schedule future event(s)

    # e.g.,
    # sim <- scheduleEvent(sim, time(sim) + increment, "projectSpeciesDist", "t

    # ! ----- STOP EDITING ----- ! #
  },
  warning(noEventWarning(sim))
)
return(invisible(sim))
}

### template initialization
Init <- function(sim) {
  # # ! ----- EDIT BELOW ----- ! #

  # ! ----- STOP EDITING ----- ! #

  return(invisible(sim))
}

### template for save events
Save <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event
  sim <- saveFiles(sim)

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

```

```

}

### template for plot events
plotFun <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # do stuff for this event
  sampleData <- data.frame("TheSample" = sample(1:10, replace = TRUE))
  Plots(sampleData, fn = ggplotFn) # needs ggplot2

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

### template for your event1
Event1 <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # THE NEXT TWO LINES ARE FOR DUMMY UNIT TESTS; CHANGE OR DELETE THEM.
  # sim$event1Test1 <- " this is test for event 1. " # for dummy unit test
  # sim$event1Test2 <- 999 # for dummy unit test

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

### template for your event2
Event2 <- function(sim) {
  # ! ----- EDIT BELOW ----- ! #
  # THE NEXT TWO LINES ARE FOR DUMMY UNIT TESTS; CHANGE OR DELETE THEM.
  # sim$event2Test1 <- " this is test for event 2. " # for dummy unit test
  # sim$event2Test2 <- 777 # for dummy unit test

  # ! ----- STOP EDITING ----- ! #
  return(invisible(sim))
}

.inputObjects <- function(sim) {
  # Any code written here will be run during the simInit for the purpose of creating
  # any objects required by this module and identified in the inputObjects element of def.
  # This is useful if there is something required before simulation to produce the module

```

```

# object dependencies, including such things as downloading default datasets, e
# downloadData("LCC2005", modulePath(sim)).
# Nothing should be created here that does not create a named object in inputO
# Any other initiation procedures should be put in "init" eventType of the doEv
# Note: the module developer can check if an object is 'suppliedElsewhere' to
# selectively skip unnecessary steps because the user has provided those input
# simInit call, or another module will supply or has supplied it. e.g.,
# if (!suppliedElsewhere('defaultColor', sim)) {
#   sim$map <- Cache(prepareInputs, extractURL('map')) # download, extract, load
# }

#cacheTags <- c(currentModule(sim), "function:.inputObjects") ## uncomment this
dPath <- asPath(getOption("reproducible.destinationPath", dataPath(sim)), 1)
message(currentModule(sim), ": using dataPath '", dPath, "'.")

# ! ----- EDIT BELOW ----- ! #

# ! ----- STOP EDITING ----- ! #
return(invisible(sim))
}

ggplotFn <- function(data, ...) {
  ggplot2::ggplot(data, ggplot2::aes(TheSample)) +
    ggplot2::geom_histogram(...)
}

```

We draw your attention to:

- As we said earlier, we could have added yearly projected values to the `sppAbundanceDT` table. In this case we probably would have changed this object's name, since `MaxEnt` is not modelling species abundance, but probability of occurrence. We suggest this as an exercise to do on your own.
- How links with the data modules are established by declaring data modules' output objects as expected inputs for this module.
- How the `fitSDM` event does not schedule itself (the `SDM` only needs to be fitted once).
- How, unlike `fitSDM`, the `projSDM` event schedules itself so that model

projections are repeated for each year, provided that there is corresponding environmental data – notice how the functions `fitSDMEvent` and `projSDMEvent` both check that there is data for the current year of the workflow run (`time(sim)`).

- How the fitted model object (`sdmOut`) and its evaluation (`evalOut`) are both module outputs. This way these objects can not only be used by other events, but also inspected by the user after the workflow run is finished⁸.

2.2.3 Additional module scripts and functions

All functions needed by a module can be coded within the module script – e.g. all the main “event functions” and before the `.inputObjects` function. However, their exact location is up to the developer.

For long, complex modules involving many (potentially long) functions, scripting all functions within a module’s script quickly renders it too long and hard to maintain. Instead, we recommend that additional (i.e., non-event) functions are put into separate scripts and in the module’s `R/` folder. *SpaDES* automatically sources any scripts contained in this folder and associates its objects/functions with their respective module.

! Note that, functions/objects defined in the modules’ `.R` scripts, or in `.R` scripts in the module `R/` folder are *not shared across modules*. If many modules need use the same functions, consider:

- putting the functions in an R package (in CRAN or GitHub) added to the modules’ metadata and installed as other packages;
- (less ideally) putting the functions in a *local* R package that is installed (using, e.g., `devtools::install_local`) or loaded (using, e.g. `devtools::load_all("<pathToPackage>")`) by `setupProject` via, e.g., the `sideEffects` argument.

Avoid repeating the same function definitions across module scripts or R/ folders, as this makes maintenance and error fixing harder.

In this example, we defined the plotting functions called by `Plots` into separate scripts in the modules’ `R/` folders. Modules `speciesAbundanceData` and `projectSpeciesDist` both have the function `plotSpatRaster` in a

⁸See [Transparent models](#).

plotSpatRaster.R script saved in their respective R/ folders (this is not great, but creating a package for this example was overkill):

```
#' Function used to plot SpatRaster
#'
#' To be used with Plots
#'
#' @param ras a SpatRaster layer
#' @param title character. Plot title
#' @param xlab character. X-axis title
#' @param ylab character. Y-axis title
#'
#' @importFrom rasterVis gplot
#' @importFrom ggplot2 geom_tile scale_fill_brewer coord_equal theme_bw

plotSpatRaster <- function(ras, plotTitle = "", xlab = "x", ylab = "y") {
  gplot(ras) +
    geom_tile(aes(fill = value)) +
    scale_fill_distiller(palette = "Blues", direction = 1,
                        na.value = "grey90", limits = c(0,1) ) +
    theme_classic() +
    coord_equal() +
    labs(title = plotTitle, x = xlab, y = ylab)
}
```

The climateData module uses the function plotSparRasterStk, which is in the plotSparRasterStk.R in its R/ folder:

```
#' Function used to plot SpatRaster Stacks
#'
#' To be used with Plots
#'
#' @param stk a SpatRaster stack.
#' @param title character. Plot title
#' @param xlab character. X-axis title
#' @param ylab character. Y-axis title
#'
#' @importFrom rasterVis gplot
#' @importFrom ggplot2 geom_tile facet_wrap scale_fill_brewer coord_equal

plotSpatRasterStk <- function(stk, plotTitle = "", xlab = "x", ylab = "y") {
```

```
gplot(stk) +  
  geom_tile(aes(fill = value)) +  
  scale_fill_distiller(palette = "Blues", direction = 1, na.value = "grey90") +  
  theme_classic() +  
  coord_equal() +  
  facet_wrap(~ variable) +  
  labs(title = plotTitle, x = xlab, y = ylab)  
}
```

Go ahead and create these scripts with the functions above and put them in the right R/ folders.

2.3 Running the model

Note that the `dismo` package (a dependency of the `projectSpeciesDist` module) needs `rJava` to run `maxent`. In turn, `rJava` needs a working Java installation. We attempt to warn the user about problems loading `rJava`, which are usually related to Java not being found on the system.

Please make sure Java is installed on your machine before proceeding (Java can be downloaded [here](#)). If you can't install Java (e.g. no admin rights) you will still be able to run the using the generalised linear model (GLM) option.

2.3.1 Workflow set-up

After the modules are created, we go back to the `Part2_SDMs.R` script to set up and run the simulation.

We will use `SpaDES.project::setupProject` to create the project folder tree and all necessary arguments to initialise the workflow with `simInit`.

Briefly, `setupProject` will create a full project folder structure (it will add folders to `projPath` like `inputs/` and `outputs/`) and an [RStudio Project](#) there. It will also look at all package dependencies listed in the modules and install them if necessary, amongst many other things (see e.g. `sideEffects`).

Basically, `setupProject` creates a list of arguments that `simInit` needs to initialise the workflow/simulation:

- I also sets global options, e.g.:

- See `?setupProject` for a full description of it's capabilities and [Workflows with setupProject](#) for more information.

```
library(SpaDES.project)

mySimMaxEnt <- setupProject(paths = list("packagePath" = "packages/",
                                         "projectPath" = projPath),
                           Restart = TRUE,
                           modules = c("speciesAbundanceData", "climateData", "p"),
                           options = list(reproducible.useCache = TRUE,
                                           reproducible.cachePath = paths$cachePath,
                                           reproducible.destinationPath = paths$destinationPath,
                                           spades.moduleCodeChecks = FALSE,
                                           repos = repos),
                           packages = c(
                             "DiagrammeR",
                             "ggpubr",
                             "SpaDES.experiment",
                             "SpaDES.tools"
                           ),
                           params = list(
                             "speciesAbundanceData" = list(
                               ".plots" = c("png"),
```

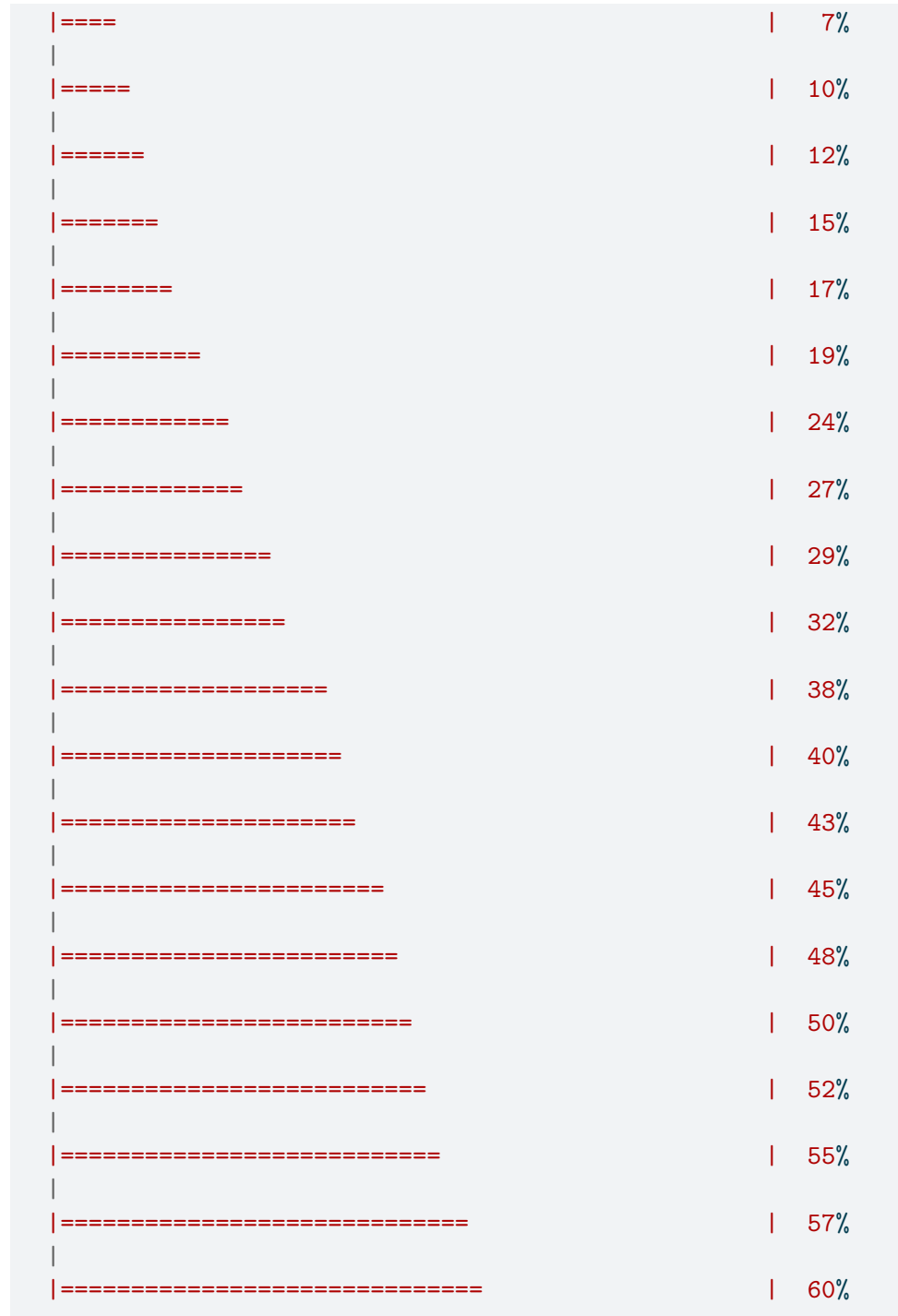
```

        ".useCache" = c(".inputObjects", "init")
    ),
    "climateData" = list(
        ".plots" = c("png"),
        ".useCache" = c(".inputObjects", "init")
    ),
    "projectSpeciesDist" = list(
        "statModel" = "MaxEnt",
        ".plots" = c("png"),
        ".useCache" = c(".inputObjects", "init")
    )
),
times = list(start = 1, end = 5, timeunit = "year"),
studyAreaRas = {
    studyArea <- SpaDES.tools::randomStudyArea(size = 1e10, seed = 123456789)
    terra::rasterize(studyArea,
        terra::rast(extent = terra::ext(studyArea),
            crs = terra::crs(studyArea, prj4 = "WGS84",
                resolution = 1000))
},
sideEffects = {
    ## dismo needs a few tweaks
    out <- reproducible::preProcess(targetFile = "maxent.jar",
        url = "https://github.com/mjendryk/maxent.jar",
        destinationPath = paths$input,
        fun = NA)

    file.copy(out$targetFilePath,
        file.path(system.file("java", package = "dismo"),
            overwrite = TRUE)
}

)
##
|
|
|
|=
|
|==
|

```



```

|
|=====| 62%
|
|=====| 65%
|
|=====| 67%
|
|=====| 69%
|
|=====| 72%
|
|=====| 74%
|
|=====| 76%
|
|=====| 83%
|
|=====| 85%
|
|=====| 93%
|
|=====| 95%
|
|=====| 100%

## check that rJava can be loaded.
if (!require(rJava, quietly = TRUE)) {
  stop(paste("Your Java installation may have problems, please check.\n",
    "See https://www.java.com/en/download/manual.jsp for Java installation"))
}

```

A few things to note about the code above:

- In the `options` argument, notice the use of `paths` (an internal object that will become a slot in `mySimMaxEnt`) to refer to the cache (`cachePath`) and inputs (`inputPath`) folders created by `setupProject`;
- `paths$inputPath` sets the directory to where all downloaded and pre-processed data goes;

- `Restart = TRUE` opens a new RStudio window (and a clean R session). This means that the `setupProject()` call and code prior to it needs to be run again;
- We use the `packages` argument to list extra packages that are not module dependencies but are needed externally (e.g., for plotting after the workflow run); none of these is loaded;
- The `studyAreaRas` is internally created from a random polygon drawn in SW Alberta, Canada, using `SpaDES.tools::randomStudyArea`, by executing the code chunk passed in the `{ }` (Figure 2.1).

⚠ Package installation issues

Windows is especially prone to package installation issues, especially when using RStudio with default *Global Options*.

If you face issues installing packages, this [troubleshooting guide](#) can help. If using RStudio, pay special attention to “[Package has already been loaded](#)” section.

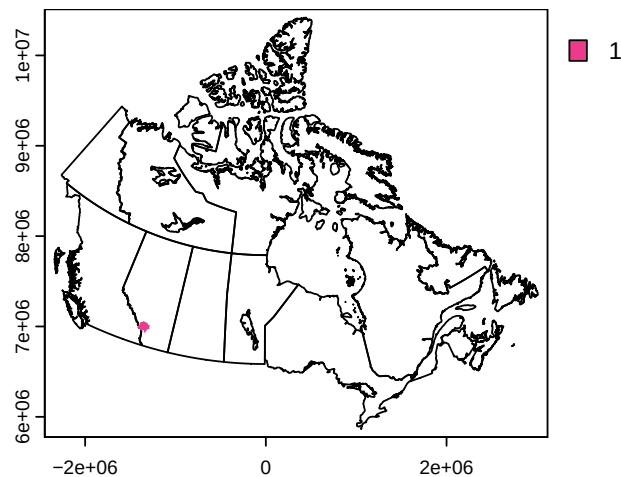


Figure 2.1: Study area within Canada.

Now we can create a very similar list for the workflow using the GLM mod-

elling approach. All we do is create a copy of `mySimMaxEnt` and change a parameter value:

```
mySimGLM <- mySimMaxEnt
mySimGLM$params$projectSpeciesDist$statModel <- "GLM"
```

2.3.2 Workflow initialisation

SpaDES workflows use the `simList` object which can only be created using `SpaDES.core::simInit`. `simInit` also runs all the `.inputObjects()` functions of the modules being used to prepare default inputs supplied by the developer. `simInit2` is a wrapper around `simInit` that allows directly passing the list output by `setupProject`.

```
simInitMaxEnt <- SpaDES.core::simInit2(mySimMaxEnt)
simInitGLM <- SpaDES.core::simInit2(mySimGLM)
```

i `SpaDES.core::simInit()`, `SpaDES.core::simInit2()`,
`SpaDES.core::spades()`, `SpaDES.core::simInitAndSpades()`
and `SpaDES.core::simInitAndSpades2()`

- `simInit()` and `simInit2()` are identical in that they create the `simList` object, but `simInit2()` accepts a list of named arguments, such as the output of `setupProject()`⁹.
- `spades()` takes the `simList` object and executes the workflow (i.e. schedules and runs module events);
- `simInitAndSpades()` is a wrapper function that runs `simInit()` followed by `SpaDES.core::spades()` (which executes the workflow);
- `simInitAndSpades2()` is equivalent to `simInitAndSpades()` but accepts a list of named arguments, such as the output of `setupProject()`.

Before running the simulation we look at the module linkage diagrams produced by `moduleDiagram` (Figure 2.2) and `objectDiagram` (Figure 2.3) to assess whether modules are linked as expected.

⁹See Section 1.2.5.1 for an example using `simInit()`. See Chapter 7 - The `simList` - What is it? of our book [*Robust and nimble scientific workflows, using SpaDES*](https://predictiveecology.org/training/_book/simList.html) for more detail on the `simList` object.

```
SpaDES.core::moduleDiagram(simInitMaxEnt)  
SpaDES.core::objectDiagram(simInitMaxEnt)
```

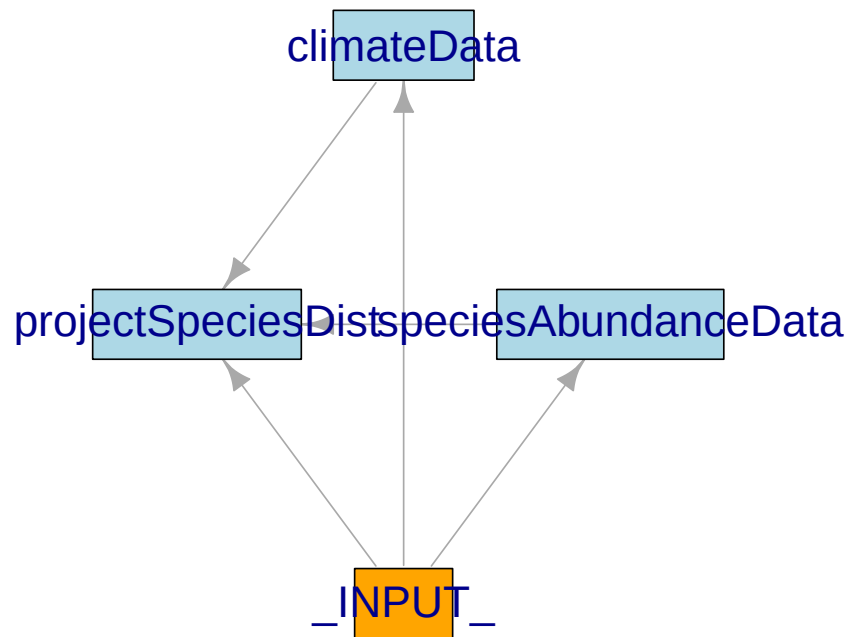


Figure 2.2: Module network diagram.

2.3.3 Workflow runs

To run the workflow, we can call `spades` on the `simLists` (`simInitMaxEnt` and `simInitGLM`) generated by `simInit`, or use `experiment2` from the `SpaDES.experiment` package.

`experiment2` will run as many simulations as `simLists` X repetitions, and or-

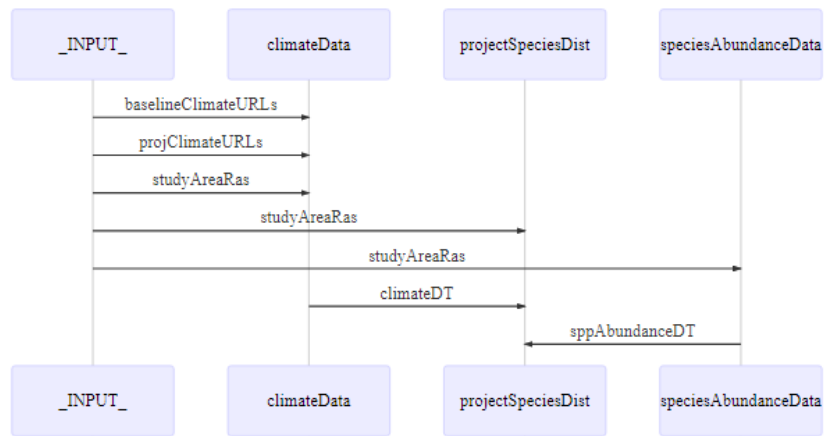


Figure 2.3: Module diagram showing module inter-dependencies with object names.

ganise outputs into sub-folders within the `SpaDES.core::outputPath(<simList>)` directory (which you can access using `mySimMaxEnt$paths$outputs`). It can also parallelise replicate runs using `future`. See `?experiment2` for examples.

We advise using `spades` when running the model for the first time. Passing the argument `debug = TRUE` will print the progress of the simulation in detail. This helps diagnosing problems when the simulation fails, but also seeing which events are being executed and when particular cache calls are activated.

```
## run simulation
quickPlot::clearPlot(force = TRUE)    ## wipe the graphics device and open a new g

## This runs one simulation and stores outputs in the main 'outputs' folder
## - not what we want, but good for testing
# mySimOut <- spades(simInitMaxEnt, debug = TRUE)

## Better to use when spades runs error-free on the simLists
myExperiment <- experiment2(MaxEnt = simInitMaxEnt,
                             GLM = simInitGLM,
                             debug = TRUE,
                             replicates = 1,
                             clearSimEnv = FALSE)    ## prevent removing objects f

## save outputs
qs::qsave(myExperiment, file.path(SpaDES.core::outputPath(simInitMaxEnt), paste0
```

Try to execute the `spades` call twice to see how much faster it runs after many of the operations have been cached (see also [Caching](#)).

By default the data modules (*speciesAbundanceData* and *climateData*) save figures of the input species and climate layers (Figure 2.4 and Figure 2.5 , respectively).

The prediction module also outputs the projections for each climate period automatically (Figure 2.6 and Figure 2.7).

The projected layers can also be accessed and plotted via the `simList` object, as can the model validation results.

From the results we can see that the MaxEnt and GLM predictions do not seem to agree, indicating a potential problem. We may be missing important covariates, interactions, or simply more appropriate algorithms.

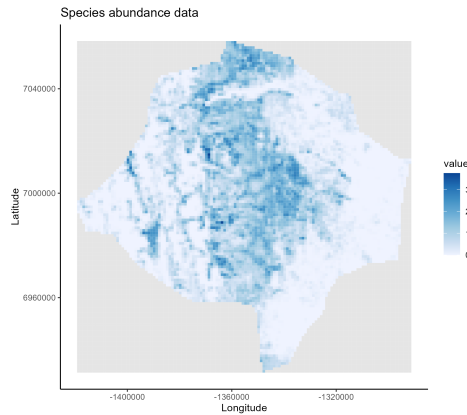


Figure 2.4: Prediction plots. Input *Picea glauca* percent cover across the landscape. Note that values are converted to presence/absence.

Peruse each model’s estimated coefficients and residuals, and validation results will be a good first step to diagnosing the problem.

```
tryCatch(myExperiment$MaxEnt_rep1$sdmOut) ## this links to an html page
plot(myExperiment$MaxEnt_rep1$sdmOut)

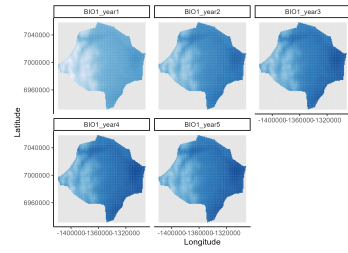
sets <- par(mfrow = c(2,2))
plot(myExperiment$GLM_rep1$sdmOut)
par(sets)

## check validation results for the two models
myExperiment$MaxEnt_rep1$evalOut
myExperiment$GLM_rep1$evalOut
```

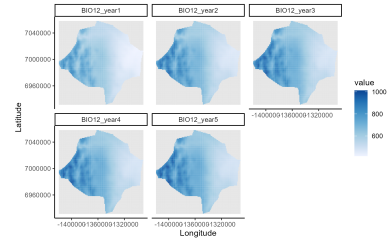
2.3.3.1 Adding a new climate scenario

Because data were linked to the modules (and the forecasting) via the modules’ metadata and inputs, adding a new climate scenario and re-running forecasts is easy.

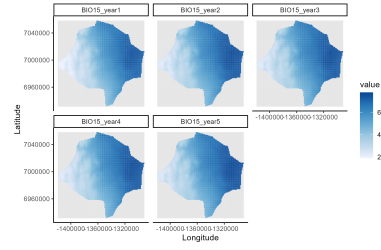
To do so, we need only to change the URLs for the climate layers, by passing a custom `projClimateURLs` data.table to the `climateData` module. `SpaDES` will take care of downloading and processing the new layers, as well as



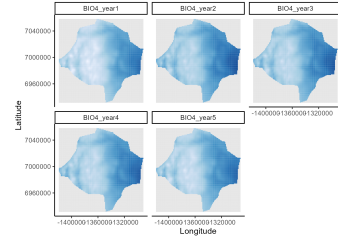
(a)



(b)



(c)



(d)

Figure 2.5: Prediction plots. Bioclimatic variables under baseline (year 1) and future conditions.

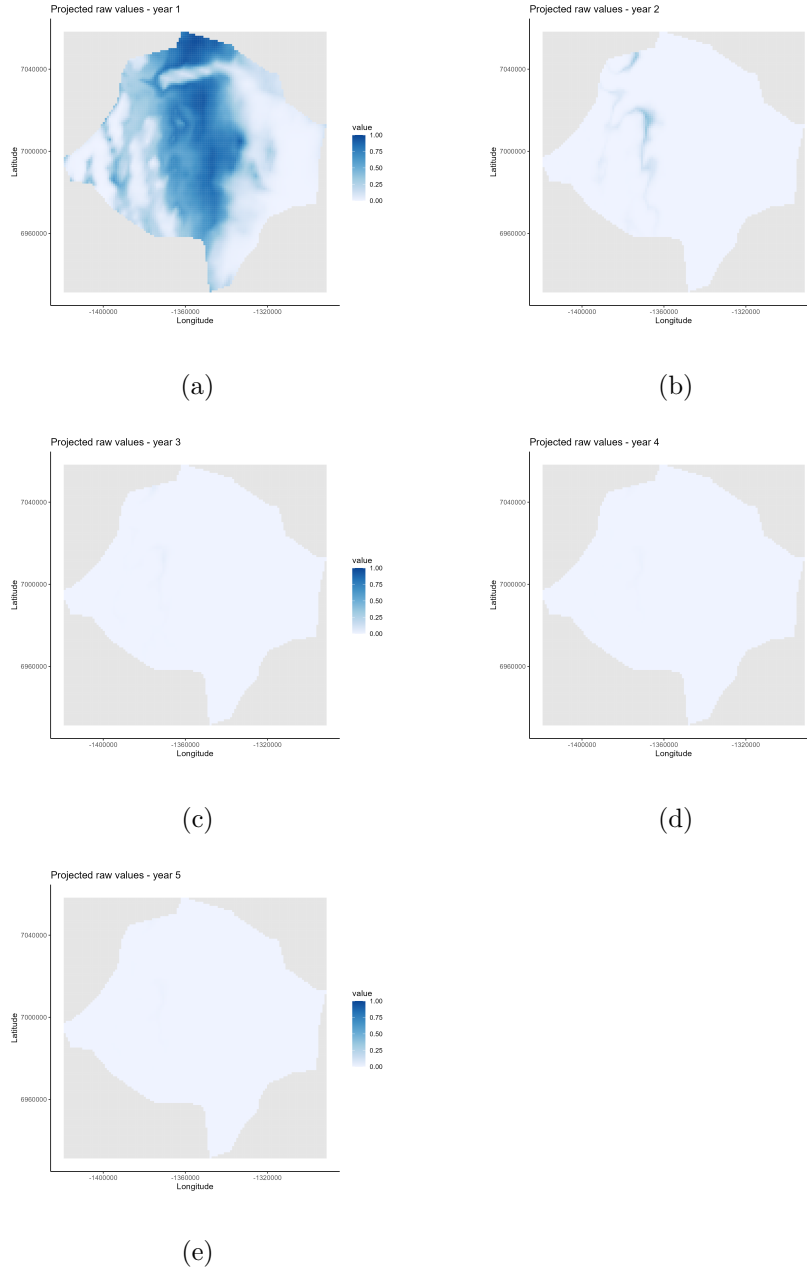


Figure 2.6: Prediction plots: Raw predicted values of species probability of occurrence under (left to right) baseline climate conditions (first year of simulation), 2021-2040, 2041-2060, 2061-2080 and 2081-2100 climate conditions (second to fifth years of simulation) - using MaxEnt.

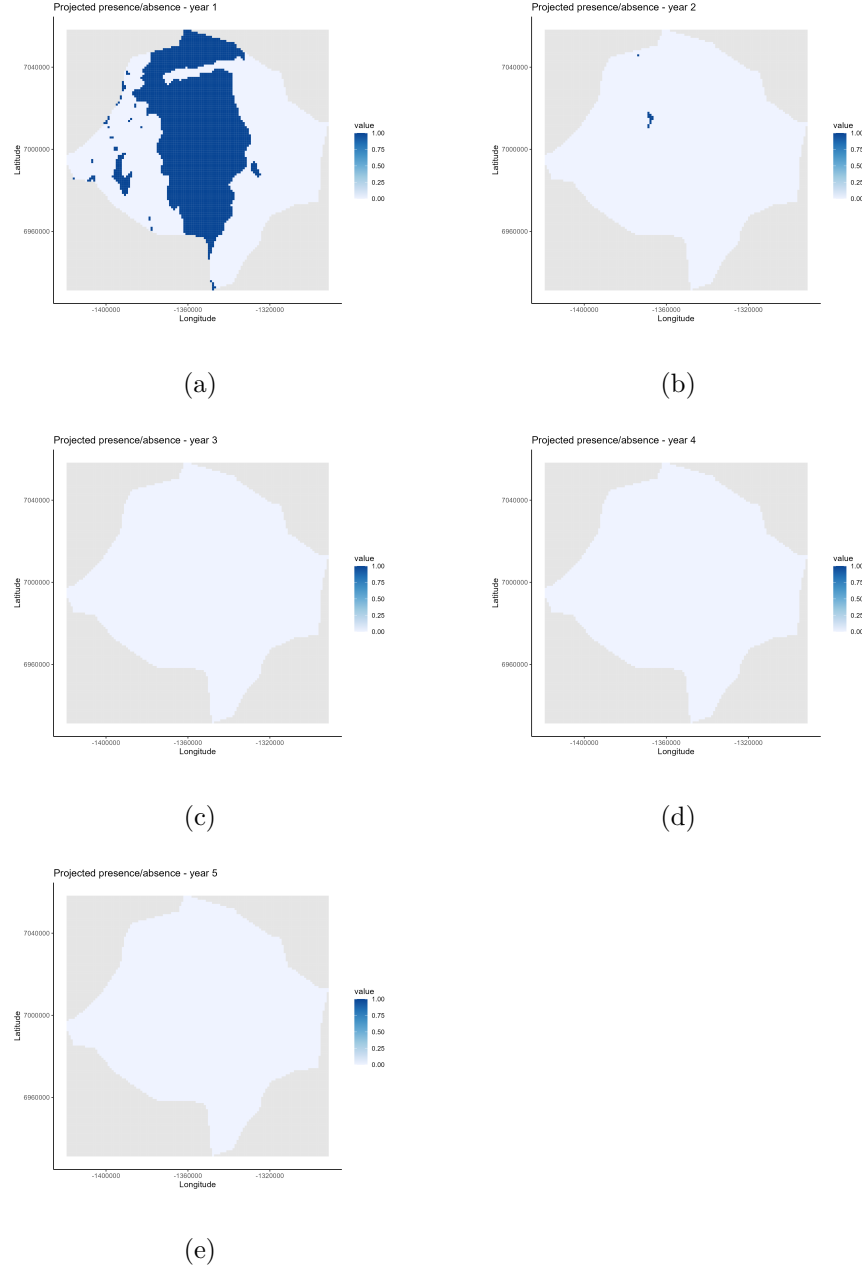


Figure 2.7: Prediction plots: Predictions of *Picea glauca* presence/absence under (left to right) baseline climate conditions (first year of simulation), 2021-2040, 2041-2060, 2061-2080 and 2081-2100 climate conditions (second to fifth years of simulation) - using MaxEnt.

forecasting.

Model fitting will also be repeated, even if the baseline data did not change, because the `kfold` function we use to partition the data into the training and testing subsets randomly assigns cases to each group. If this is not desired, we can set a random seed before running the fitting event (`fitSDM`) by passing the `.seed` parameter to the *projectSpeciesDist* module (e.g., `.seed = list("fitSDM" = 123)`).

```
## Run with another climate scenario - the most contrasting scenario to SSP 585.
## Make copies of the outputs of setupProject, then get the original table from
## one of the simulations, and replace the climate scenario.
mySimMaxEnt2 <- mySimMaxEnt
mySimGLM2 <- mySimGLM
projClimateURLsNew <- copy(myExperiment$MaxEnt_rep1$projClimateURLs)
projClimateURLsNew[, `:=`(URL = gsub("ssp585", "ssp126", URL),
                               targetFile = gsub("ssp585", "ssp126", targetFile))]

mySimMaxEnt2$projClimateURLs <- projClimateURLsNew
mySimGLM2$projClimateURLs <- projClimateURLsNew

## now initialise the workflows using the new scenarios
simInitMaxEnt2 <- SpaDES.core::simInit2(mySimMaxEnt2)
simInitGLM2 <- SpaDES.core::simInit2(mySimGLM2)

myExperiment2 <- experiment2(MaxEnt2 = simInitMaxEnt2,
                             GLM2 = simInitGLM2,
                             debug = TRUE,
                             replicates = 1,
                             clearSimEnv = FALSE)

## save outputs
qs::qsave(myExperiment2, file.path(SpaDES.core::outputPath(mySimMaxEnt2), paste0("myExper
```

2.3.3.2 Proposed exercises

1. try changing the climate layers (e.g., use different climate scenarios or General Circulation models) and rerunning predictions;
2. try adding other statistical algorithms;
3. try breaking up the prediction module into three modules: a calibra-

tion module, a prediction module and a validation module.

Have fun!

2.3.3.3 Making use of `simList` for reporting

Another advantage of having all simulation parameters, inputs and outputs centralised in one object, is that we can easily inspect and manipulated them afterwards, without the need to load separate objects back into R.

Here we show how we capitalize on this `SpaDES` feature to create figures of the outputs (Figure 2.8).

```
## MaxEnt predictions across time and for each climate scenario -----
## combine plots from two distinct simulations in a single figure
## (the same can be done to compare MaxEnt and GLM, or plot all projections)

## fetch the internal plotting function rom any of the two simulations,
## instead of repeating code here
plotFun <- myExperiment$MaxEnt_rep1@.envir$.mods$climateData$plotSpatRasterStk

## raw predictions exported by the module
sppDistProjMaxEnt <- myExperiment$MaxEnt_rep1$sppDistProj
sppDistProjMaxEnt2 <- myExperiment2$MaxEnt2_rep1$sppDistProj

## we convert the raw predictions into presence absence
## using exported threshold
sppDistProjMaxEnt_PA <- as.int(myExperiment$MaxEnt_rep1$sppDistProj > myExperimen
sppDistProjMaxEnt2_PA <- as.int(myExperiment2$MaxEnt2_rep1$sppDistProj > myExper

## rename layers from plotting
names(sppDistProjMaxEnt) <- names(sppDistProjMaxEnt2) <- c("2001", "2021-2040",
names(sppDistProjMaxEnt_PA) <- names(sppDistProjMaxEnt2_PA) <- c("2001", "2021-20

## for a simpler plot choose only years 2001, 2041-2060 and 2081-2100
yrs <- c("2001", "2041-2060", "2081-2100")
plotMaxEnt <- plotFun(sppDistProjMaxEnt[[yrs]],
                      xlab = "Longitude", y = "Latitude",
                      plotTitle = "MaxEnt raw predictions - SSP 585") +
  scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)
plotMaxEnt2 <- plotFun(sppDistProjMaxEnt2[[yrs]],
```

```

        xlab = "Longitude", y = "Latitude",
        plotTitle = "MaxEnt raw predictions - SSP 126") +
    scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)
plotMaxEnt_PA <- plotFun(sppDistProjMaxEnt_PA[[yrs]],
        xlab = "Longitude", y = "Latitude",
        plotTitle = "MaxEnt presence/absence - SSP 585") +
    scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)
plotMaxEnt2_PA <- plotFun(sppDistProjMaxEnt2_PA[[yrs]],
        xlab = "Longitude", y = "Latitude",
        plotTitle = "MaxEnt presence/absence - SSP 126") +
    scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)

## organise the plots with mildest scenario first
## It is clear that MaxEnt and GLM do not agree in their prediction
plotAll <- ggpubr::ggarrange(
  plotMaxEnt2 +
    labs(title = expression(bold("Scenario - SSP 126")),
         y = expression(atop(bold("Raw predictions"), "Latitude")))) +
  theme(legend.title = element_blank(), legend.key.height = unit(3, "lines"),
        plot.title = element_text(hjust = 0.5), plot.margin = margin(0,0,0,0)),
  plotMaxEnt +
    labs(title = expression(bold("Scenario - SSP 585")),
         y = expression(atop(bold(""), ""))) +
  theme(plot.title = element_text(hjust = 0.5), plot.margin = margin(0,0,0,0)),
  plotMaxEnt2_PA +
    labs(title = expression(bold(""))),
         y = expression(atop(bold("Presence/absence"), "Latitude")))) +
  theme(plot.margin = margin(0,0,0,0)),
  plotMaxEnt_PA +
    labs(title = expression(bold(""))),
         y = expression(atop(bold(""), ""))) +
  theme(plot.margin = margin(0,0,0,0)),
  legend = "right", common.legend = TRUE, labels = c("a)", "b)", "c)", "d)"))

## save figure:
figDir <- checkPath(file.path(mySimMaxEnt2$paths$outputPath, "generalFigures"), create = TRUE)
ggsave(file.path(figDir, "MaxEntPredictions.png"), width = 13.5, height = 5.5, units = "inches")

```

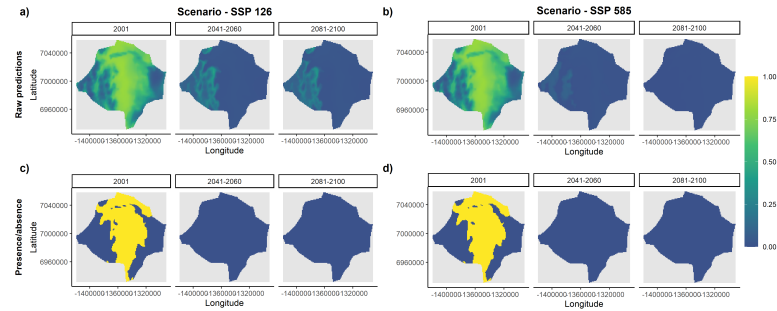


Figure 2.8: Adding a new scenario: Predictions of *Picea glauca* probabilities of presences and presence/absence under (left to right) baseline climate conditions, 2041-2060, and 2081-2100 climate projections under two emission scenarios (SSP 136 and SSP 585, the default) – showing MaxEnt forecasts only.

2.4 Caching

In this example, we relied on caching to avoid repeating computationally intensive or long operations. Running the `simInit` and `spades` calls a second time (even after restarting R session) was faster and `SpaDES` informed us of instances where cached objects were being retrieved:

```
(...)
Mar05 19:56:53 clmtDt 1 climateData init 1\
Mar05 19:56:53 clmtDt ...(Object to retrieve (a7816e2d0deb3b29.rds)) Mar05 19:56
(...)
```

Caching in `SpaDES` is managed by the `reproducible` package, and can be generally broken down into two types: explicitly coded by the module developer, or internal to `SpaDES` functions.

2.4.1 Explicitly caching operations

Throughout the data modules we explicitly cached several data preparation operations using the functions `Cache` and `prepInputs` from the `reproducible` package.

In brief, `Cache` searches for a stored (i.e. cached) output of a given function call; if it does not find it, `Cache` executes the function call, saves its output and saves information about the function inputs and the function's code. If it does find it, `Cache` compares the present inputs and function code

against their cached counterparts. In case of a mismatch, the function call is executed again and re-cached.

`prepInputs` calls `Cache` internally at several points, notably to cache spatial processing tasks (e.g. projecting and cropping spatial layers to a study area raster). Another great feature of `prepInputs` is that when it has a source URL for the target file (as when we used `prepInputs` to download species % cover and climate layers), it first checks whether the data have already been downloaded (and potentially extracted from an archive folder – `.zip` file). This is not the same thing as caching, but also avoids unnecessary downloads that can be time consuming.

Note that caching operations involving stochasticity should be avoided, as it will prevent new random outputs from being generated.

We recommend exploring the examples available in the `Cache` and `prepInputs` R documentation to learn more about their capabilities. In particular, read about `showCache`, `clearCache` and the argument `userTags`, which allow consulting and deleting cached files.

 Warning

Cache does not deal well with the `apply` family of functions, which is why we used `Map` (instead of `mapply`) to iteratively apply `prepInputs` to several climate layer URLs.

2.4.2 Implicit caching of events

`SpaDES` offers implicit caching of events via the global parameter `.useCache`, which comes in the template generated by `newModule`. We call this “implicit” caching, because the developer does not need to add any caching mechanisms to the module code. `SpaDES` automatically reads the value of the `.useCache` parameter and activates caching in the module accordingly.

This parameter can be used to cache (or not) all or some module events (in their entirety). In our example, we cached the the `.inputObjects` function and or `init` event, but not other events of the `projectSpeciesDist` module. Setting `.useCache = FALSE` would turn of event caching completely and `.useCache = TRUE` would cache all events.

Notice how loading cached events (last line below) produces a slightly different message from loading other cached operations (second line below):

```

Mar05 19:58:34 spcsbn 1 speciesAbundanceData init 1
Mar05 19:58:34 spcsbn ...(Object to retrieve (bffb48cc055c846.rds))
Mar05 19:58:35 spcsbn loaded cached copy of init event in speciesAbundanceData m

```

2.4.3 Controlling caching without changing module code

The user can turn caching on/off without changing module code via three different mechanisms:

- via the `.useCache` parameter – as explained above (**Implicit caching of events**), setting this parameter controls event caching inside a module;
- via `options("reproducible.useCache")` – setting this option to `TRUE` or `FALSE` in the global environment (`.GlobalEnv`) will affect *all* caching (inside and outside Spades modules and the simulation);
- via the argument `spades(.useCache = ...)` – this argument behaves in the same way as the `.useCache` module parameter, but supersedes it across *all* modules (i.e. if `spades(..., .useCache = FALSE)`, caching will be turned off even if a module's `.useCache` is `TRUE`).

2.5 Best practices

2.5.1 Reproducible package installation

When sharing code, it is good practice to provide other users with a list of necessary packages (e.g. by listing the sequence of `library` calls at the start of a script). We go a step further and advise users to provide code that automatically installs all necessary packages at the start of their controller script. In addition, all modules should contain a full list of packages that they depend on, and any particular versions necessary.

In the example above, we rely on `setupProject` to install all module package dependencies and a few extra ones¹⁰

If not using `setupProject`¹¹ `options("spades.useRequire")` is set to `TRUE`

¹⁰Via the `setupProject(..., packages)` argument.

¹¹I.e., if creating the required inputs for `simInit` “manually” on the global environment and passing them to `simInit()`. We, Spades developers, are gradually moving away from this practice to keep the global environment as “clean” as possible. Robust and nimble scientific workflows, using Spades - Best practices.

(the default), SpaDES will automatically attempt to install any packages listed across all modules if they are not installed in `.libPaths()`, or if the installed version (or branch if installing from GitHub) does not correspond to what is listed in the module `.R` script. Users can also use `Require::pkgSnapshot()` to save a list of installed packages that can be used later by `Require` to install all necessary packages in another machine:

```
Require::pkgSnapshot("pkgsnapshot.txt", libPaths = .libPaths()[1]) ## you should make su

## on another machine:
Require::Require(packageVersionFile = "pkgsnapshot.txt")

## See ?Require::pkgSnapshot() for more examples.
```

! Important

Package installation should be done as much as possible from a *clean R session* especially in the context of a SpaDES-based project, where each module can potentially have many different dependencies, which have dependencies of their own (notice how we didn't load any packages other than `SpaDES.project`¹².

2.5.2 Protect yourself and others from common mistakes/problems

A developer should put in place code checks, warnings and messages that protect and warn the user against common mistakes or issues. Some of these fall in the category of *code assertions* – small tests that verify a snippet of code. More complex tests that assess whether the module (or a group of modules) is producing expected results for, e.g., an ecological point of view fall in the category of *integration tests*. Here, we only talk about code assertions.

A common assertion is to verify that input format and class conform to what the function expects. If this is not the case, the developer may add a mechanism to correct the faulty inputs (potentially with a warning or message telling the user it did so) or simply stop the computations with a meaningful error. We provide two examples in the `climateData` module,

¹²See also [Robust and nimble scientific workflows, using SpaDES - 3 Troubleshooting R package installation](#).

where the `climateInit` function checks whether the bioclimatic variable names are consistent between the baseline and projected climate data, and whether their raster layers match.

Other assertions can prevent undesirable function behaviours, such as the `if` statement protecting the `newModule` call in `Part2_SDMs.R`, or warn the user that something is missing early on, such as the check for `studyAreaRas` existence in the `.inputObjects` of the data modules).

Bear in mind that these are just examples assertions and integration tests are as diverse as the code they test.

2.5.3 Readable code

There are several guides on how to write reader-friendly code. Even if the developer is forever the sole reader of their own code, there are benefits to writing readable code. First, working on it is less tiresome. Second, we quickly forget why we wrote code in a certain way. Code that is well documented and readable is easier to “come back to” and adapt.

We follow many of the [recommendations by Hadley Wickham](#), and highlight below those that we find particularly important:

- spacing around operators;
- spacing before left parenthesis, except in a function call;
- adding curly braces after `if`, `else`, `for` and `function`, unless they are very short statements;
- thoroughly commenting the code;
- naming functions meaningfully and avoiding to re-use function names (e.g. avoid `c <- function (...) {}`, as `c` is already a `base` function).

You can automatically cleanup and format your code using the `styler` package. This package provides an Rstudio addin to easily style a block of selected code, or an entire file.

2.5.4 Module documentation (`.Rmd` file)

The `newModule` also creates a template module `.Rmd` file that is meant to document the module. The template suggests a few key sections that should be part of any module’s documentation. Notably, an overview of the module

and of its inputs, parameters, outputs and general event flow, together with more in-depth descriptions of each of these sections.

The documentation may also contain reproducible examples of how a module can be used, although this is not always relevant. For instance, data modules are often meaningless without downstream modules that use their outputs.

See the manual of our forest landscape simulation model [LandR Biomass_core](#), as an example of how we document some of our SpaDES modules.

2.5.5 Coding for the future

We often make coding decisions that we regret a few months down the line. This is why as module developers, it is a good idea to think about other possible applications of a module or potential expansion avenues.

For instance, trying to imagine if the module can be scaled up or transferred to different study areas, may influence the format of expected inputs and of outputs. In our example, we exported the same type of information (species % cover and climate data) as raster layers and as tables, because we could foresee that the tables could be used to store several projections in a more compact format.

2.5.6 Transparent models

Model transparency is not only about using open source code and making it available. Providing easy access to model data, parameters and outputs is also important. For instance, in our example we deliberately exported the fitted statistical model `sdmOut`, data (`sdmData`) and evaluation statistics (`evalOut`) so that they can be more easily inspected by the user, without needing to “dive in” the code.

SpaDES also offers the ability to save any objects that are exported to the `simList` object *without having to change module code*. To do so, the user passes a `data.frame` of object names and (potentially) the simulation times when they should be saved to the `outputs` argument of `setupProject` (and `simInit`). Because objects are saved as `.rds` files by default, any object class can be saved to disk¹³.

¹³See `?SpaDES.core::outputs` for more information.

2.6 Additional notes

SpaDES is an extremely powerful family of R packages, and its potential goes well beyond what has been discussed here.

We recommend going to the [SpaDES webpage](#) and the [Robust and nimble scientific workflows, using SpaDES](#) book to learn more.

See also the [Predictive Ecology Community](#) website for news, upcoming SpaDES workshops, a list of packages, and publications, as well as the Predictive Ecology Github repository for all available SpaDES modules and SpaDES-related packages that we maintain.

2.7 Acknowledgements

We acknowledge the World Climate Research Programme, which coordinated and promoted CMIP6, and the climate modelling groups for producing and making available their model output, the Earth System Grid Federation (ESGF) for archiving the data and providing access, and WorldClim for downscaling and sharing climate projections and preparing bioclimatic variables.

Happy SpaDESing!

References

- [1] Stephen E. Fick and Robert J. Hijmans. “WorldClim 2: new 1-km spatial resolution climate surfaces for global land areas”. In: *International Journal of Climatology* 37.12 (Oct. 2017), pp. 4302–4315. issn: 0899-8418, 1097-0088. DOI: [10.1002/joc.5086](https://doi.org/10.1002/joc.5086). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/joc.5086>.
- [2] Robert J. Hijmans et al. “dismo: Species Distribution Modeling”. In: (2021). URL: <https://CRAN.R-project.org/package=dismo>.
- [3] Neil Cameron Swart et al. *CCCma CanESM5 model output prepared for CMIP6 ScenarioMIP*. Version Number: 20220228 Type: dataset DOI: 10.22033/ESGF/CMIP6.1317. 2019. DOI: [10.22033/ESGF/CMIP6.1317](https://doi.org/10.22033/ESGF/CMIP6.1317). URL: <http://cera-www.dkrz.de/WDCC/meta/CMIP6/CMIP6.ScenarioMIP.CCCma.CanESM5>.
- [4] Wilfried Thuiller et al. “biomod2: Ensemble Platform for Species Distribution Modeling”. In: (2021).

Appendix A

Complete R script to Chapter 1

```
## start from a clean R session
options(repos = c("https://predictiveecology.r-universe.dev/",
                  CRAN = "https://cloud.r-project.org"))
install.packages(c("SpaDES.project", "SpaDES.core"))

library(SpaDES.project)

projPath <- "~/SpaDES4Dummies_Part1"

## Create modules if need be
if (!dir.exists(file.path(projPath, "modules", "speciesAbundance"))) {
  SpaDES.core::newModule(name = "speciesAbundance", path = file.path(projPath, "modules"))
}

if (!dir.exists(file.path(projPath, "temperature"))) {
  SpaDES.core::newModule(name = "temperature", path = file.path(projPath, "modules"))
}

if (!dir.exists(file.path(projPath, "speciesTempLM"))) {
  SpaDES.core::newModule(name = "speciesTempLM", path = file.path(projPath, "modules"))
}
```

```
out <- setupProject(paths = list(projectPath = "~/SpaDES4Dummies_Part1",
                                packagePath = "packages"),
                   overwrite = FALSE,
                   modules = c("speciesAbundance",
                              "temperature",
                              "speciesTempLM"),
                   params = list(speciesAbundance = list(simulationTimeStep = 1,
                                                         .plotInitialTime = 1),
                                temperature = list(simulationTimeStep = 1,
                                                    .plotInitialTime = 1),
                                speciesTempLM = list(statsTimestep = 5)),
                   times = list(start = 1, end = 10, timeunit = "year"))

simInitOut <- SpaDES.core::simInit2(out)

simOut <- SpaDES.core::spades(simInitOut)
```

Appendix B

Complete R script to Chapter 2

```
if (getRversion() < "4.2.1") {  
  warning(paste("dismo::maxent may create a fatal error",  
                "when using R version < v4.2.1 and from RStudio.\n",  
                "Please upgrade R, or run this script outside of RStudio.\n",  
                "See https://github.com/rspatial/dismo/issues/13"))  
}  
  
repos <- c("https://predictiveecology.r-universe.dev/",  
          CRAN = "https://cloud.r-project.org")  
install.packages(c("SpaDES.project", "SpaDES.core"), repos = repos)  
  
## decide where you're working  
projPath <- file.path("~/SpaDES4Dummies_Part2")  
modPath <- file.path(projPath, "modules")  
  
## Let's create a self-contained module that will simulate the species' abundance for any  
if (!dir.exists(file.path(modPath, "speciesAbundanceData"))) {  
  SpaDES.core::newModule(name = "speciesAbundanceData", path = modPath)  
}  
  
if (!dir.exists(file.path(modPath, "climateData"))) {  
  SpaDES.core::newModule(name = "climateData", path = modPath)
```



```

    ),
    times = list(start = 1, end = 5, timeunit = "year"),
    studyAreaRas = {
      studyArea <- SpaDES.tools::randomStudyArea(size = 1e10, seed = 12345)
      terra::rasterize(studyArea,
        terra::rast(extent = terra::ext(studyArea),
          crs = terra::crs(studyArea, prj4 = "NAD83",
            resolution = 1000))
    },
    sideEffects = {
      ## dismo needs a few tweaks
      out <- reproducible::preProcess(targetFile = "maxent.jar",
        url = "https://github.com/mjw30/maxent.jar",
        destinationPath = paths$input,
        fun = NA)

      file.copy(out$targetFilePath,
        file.path(system.file("java", package = "dismo"),
          overwrite = TRUE)
    }
  )

  ## check that rJava can be loaded.
  if (!require(rJava, quietly = TRUE)) {
    stop(paste("Your Java installation may have problems, please check.\n",
      "See https://www.java.com/en/download/manual.jsp for Java installation"))
  }

  ## check study area:
  terra::plot(mySimMaxEnt$studyAreaRas)

  ## the GLM alternative only needs one parameter to be changed, so we can avoid
  ## repeating the `setupProject()` call
  mySimGLM <- mySimMaxEnt
  mySimGLM$params$projectSpeciesDist$statModel <- "GLM"

  ## Simulation setup - create two "simulations", one for MaxEnt another for GLM
  ## SpaDES.experiment::experiment2, will take care of subdirectories to store outputs
  simInitMaxEnt <- SpaDES.core::simInit2(mySimMaxEnt)
  simInitGLM <- SpaDES.core::simInit2(mySimGLM)

```

```

SpaDES.core::moduleDiagram(simInitMaxEnt)
SpaDES.core::objectDiagram(simInitMaxEnt)

## run simulation
quickPlot::clearPlot(force = TRUE)    ## this forces wiping the graphics device and

## This runs one simulation and stores outputs in the main 'outputs' folder
## - not what we want, but good for testing
# mySimOut <- SpaDES.core::spades(simInitMaxEnt, debug = TRUE)

## Better to use after checking that spades() runs error-free on both simLists
myExperiment <- SpaDES.experiment::experiment2(MaxEnt = simInitMaxEnt,
                                                GLM = simInitGLM,
                                                debug = TRUE,
                                                replicates = 1,
                                                clearSimEnv = FALSE)    ## prevent

## save outputs -- both simLists have the same output path, so we can use the file
qs::qsave(myExperiment, file.path(SpaDES.core::outputPath(simInitMaxEnt), paste0(

## easily check models
tryCatch(myExperiment$MaxEnt_rep1$sdmOut)    ## this links to an html page
plot(myExperiment$MaxEnt_rep1$sdmOut)

sets <- par(mfrow = c(2,2))
plot(myExperiment$GLM_rep1$sdmOut)
par(sets)

## easily check validation results for the two models
myExperiment$MaxEnt_rep1$evalOut
myExperiment$GLM_rep1$evalOut

## Run with another climate scenario - the most contrasting scenario to SSP 585.
## Make copies of the outputs of setupProject, then get the original table from
## one of the simulations, and replace the climate scenario.
mySimMaxEnt2 <- mySimMaxEnt
mySimGLM2 <- mySimGLM
projClimateURLsNew <- copy(myExperiment$MaxEnt_rep1$projClimateURLs)

```

```

projClimateURLsNew[, `:=`(URL = gsub("ssp585", "ssp126", URL),
                                targetFile = gsub("ssp585", "ssp126", targetFile))]

mySimMaxEnt2$projClimateURLs <- projClimateURLsNew
mySimGLM2$projClimateURLs <- projClimateURLsNew

## now initialise the new simulations/workflows with the new scenarios
simInitMaxEnt2 <- SpaDES.core::simInit2(mySimMaxEnt2)
simInitGLM2 <- SpaDES.core::simInit2(mySimGLM2)

myExperiment2 <- experiment2(MaxEnt2 = simInitMaxEnt2,
                             GLM2 = simInitGLM2,
                             debug = TRUE,
                             replicates = 1,
                             clearSimEnv = FALSE)

## save outputs
qs::qsave(myExperiment2, file.path(SpaDES.core::outputPath(mySimMaxEnt2), paste0("myExperiment2",
                                         "MaxEnt2", "GLM2", "rep1", "qs.qs")))

## MaxEnt predictions across time and for each climate scenario -----
## combine plots from two distinct simulations in a single figure
## (the same can be done to compare MaxEnt and GLM, or plot all projections)

## fetch the internal plotting function from any of the two simulations,
## instead of repeating code here
plotFun <- myExperiment$MaxEnt_rep1@.envir$.mods$climateData$plotSpatRasterStk

## raw predictions exported by the module
sppDistProjMaxEnt <- myExperiment$MaxEnt_rep1$sppDistProj
sppDistProjMaxEnt2 <- myExperiment2$MaxEnt2_rep1$sppDistProj

## we convert the raw predictions into presence absence
## using exported threshold
sppDistProjMaxEnt_PA <- as.int(myExperiment$MaxEnt_rep1$sppDistProj > myExperiment$MaxEnt_rep1$threshold)
sppDistProjMaxEnt2_PA <- as.int(myExperiment2$MaxEnt2_rep1$sppDistProj > myExperiment2$MaxEnt2_rep1$threshold)

## rename layers from plotting
names(sppDistProjMaxEnt) <- names(sppDistProjMaxEnt2) <- c("2001", "2021-2040", "2041-2060")
names(sppDistProjMaxEnt_PA) <- names(sppDistProjMaxEnt2_PA) <- c("2001", "2021-2040", "2041-2060")

```

```

## for a simpler plot choose only years 2001, 2041-2060 and 2081-2100
yrs <- c("2001", "2041-2060", "2081-2100")
plotMaxEnt <- plotFun(sppDistProjMaxEnt[[yrs]],
                     xlab = "Longitude", y = "Latitude",
                     plotTitle = "MaxEnt raw predictions - SSP 585") +
  scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)
plotMaxEnt2 <- plotFun(sppDistProjMaxEnt2[[yrs]],
                     xlab = "Longitude", y = "Latitude",
                     plotTitle = "MaxEnt raw predictions - SSP 126") +
  scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)
plotMaxEnt_PA <- plotFun(sppDistProjMaxEnt_PA[[yrs]],
                       xlab = "Longitude", y = "Latitude",
                       plotTitle = "MaxEnt presence/absence - SSP 585") +
  scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)
plotMaxEnt2_PA <- plotFun(sppDistProjMaxEnt2_PA[[yrs]],
                        xlab = "Longitude", y = "Latitude",
                        plotTitle = "MaxEnt presence/absence - SSP 126") +
  scale_fill_viridis_c(na.value = "grey90", limits = c(0,1), begin = 0.25)

## organise the plots with mildest scenario first
## It is clear that MaxEnt and GLM do not agree in their prediction
plotAll <- ggpubr::ggarrange(plotMaxEnt2 + labs(title = expression(bold("Scenario
                                     y = expression(atop(bold("Raw pre
                                     theme(legend.title = element_blank(), legend.key.l
                                     plot.title = element_text(hjust = 0.5), plot
plotMaxEnt + labs(title = expression(bold("Scenario
                                     y = expression(atop(bold(""), ""))),
                                     theme(plot.title = element_text(hjust = 0.5), plot
plotMaxEnt2_PA + labs(title = expression(bold("")),
                                     y = expression(atop(bold("Pres
                                     theme(plot.margin = margin(0,0,0,0)),
plotMaxEnt_PA + labs(title = expression(bold("")),
                                     y = expression(atop(bold("")),
                                     theme(plot.margin = margin(0,0,0,0)),
                                     legend = "right", common.legend = TRUE, labels = c(
figDir <- checkPath(file.path(mySimMaxEnt2$paths$outputPath, "generalFigures"), c
ggsave(file.path(figDir, "MaxEntPredictions.png"), width = 13.5, height = 5.5, un

```